SUN: Agentic Robot Policy Learning with Persistent Task Programs

Weiqi Wang[1,*], Zhi Li[1,*], Yudong Lei[2], David Martinez[1], Xiaofeng Gao[3,‡], Yuxin Jiang[1], Chenfanfu Jiang[1], Yingnian Wu[1,†], Demetri Terzopoulos[1,†], Ran Gong[4,†]

[1]University of California, Los Angeles [2]Carnegie Mellon University [3]Amazon [4]Robotics and AI Institute

***Abstract*— Model-based control can directly execute specified objectives, while learning can amortize such behaviors into reactive policies, making their combination a natural solution to multi-stage manipulation. We introduce Semantically UNified (SUN) Programs, typed executables that compile grounded relations into aligned optimal control objectives, satisfaction predicates, and learning rewards. Our harness, Kuafu, equips a foundation model as a task-level agent to orchestrate scene preparation, verification, residual RL, and data production. The agent uses program feedback to repair candidate programs and training diagnostics to calibrate relative reward weights, retaining accepted task semantics across tool calls. Across nine multi-stage manipulation tasks, Kuafu achieves 82.03% average success rate, significantly outperforming all learned baselines. Its learned controllers generate demonstrations at 10.57× the human-teleoperation rate, yielding data that improve visual-policy success by 23.6 percentage points over the strongest baseline. The policies transfer zero-shot to physical Franka and Kinova robots, demonstrating sim-to-real generalization.**

Index Terms—AI-Enabled Robotics, Integrated Planning and Learning, Reinforcement Learning

I. Introduction

Automatic multi-stage skill acquisition remains a central challenge in robot manipulation. Recent approaches leverage the code-writing capabilities of large language models to decompose tasks into motion primitives or synthesize reward functions for policy learning [1–3]. However, they face distinct limitations: motion-planning compositions often struggle with contact-rich dynamics, while reinforcement learning scales poorly to multi-stage tasks due to sparse supervision and semantic ambiguity.

Recent advances in agentic systems enable multi-step digital workflows, including web navigation [4], research and article writing [5], and task execution across desktop applications [6]. These capabilities motivate a robotics-specific execution and learning harness, where an agent can test physical task specifications and acquire deployable skills for contact-rich, multi-stage manipulation through robotics interfaces. Analogous to how a code interpreter lets agents verify software logic, a robotics harness allows them to test task descriptions under simulated physics, identify failure modes, and derive policies from verified successes.

Realizing such a harness requires mechanisms that support both rapid physics-based verification and the acquisition of deployable behaviors. Control and learning provide complementary mechanisms for these objectives: MPC enables rapid feasibility verification via closed-loop simulation [7, 8], whereas RL produces deployable reactive behaviors. An agent thus benefits from validating semantics through low-cost control verification prior to expensive policy learning.

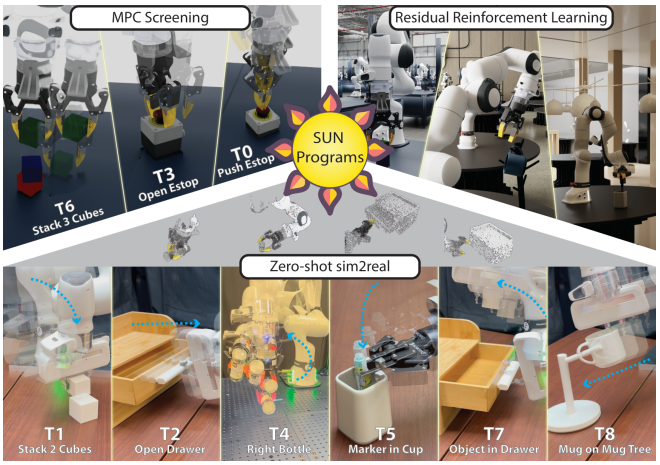


Fig. 1. Coordinated by a persistent SUN Program, the Kuafu harness equips the agent with physics-based verification, policy learning, and data-production capabilities. The program supervises the simulation controller and its demonstrations; downstream visual policies learn from recorded observations and actions alone. We evaluate nine tasks and transfer visual policies for six of them to Franka FR3 and Kinova Gen3 robots.

To this end, we introduce the *Semantically UNified (SUN) Program*: a typed, executable task specification that acts as a shared semantic contract between the control, verification, and learning tools (Figure 1). Our harness, *Kuafu*, makes this semantic contract operational through a coordinated set of robot execution and learning capabilities that the agent can invoke (Figure 2). These capabilities enable the agent to develop deployable policies from real-world videos and natural-language instructions using simulated physical and training feedback, while the SUN program preserves the verified task definition across tool calls.

Our contributions are thus twofold:

- **SUN Programs: persistent, executable task semantics.** A typed program compiles grounded relations into aligned control, monitoring, and learning interfaces, preserving the task knowledge tested by MPC across the control-to-learning handoff.
- **Kuafu: a harness for robot policy learning.** It equips the agent with scene preparation, physical verification, demonstration generation, Stage-BC, bounded residual RL, simulation evaluation, and downstream data pro-

*Equal contribution.
†Corresponding author.
‡This work does not relate to the author's position at Amazon.

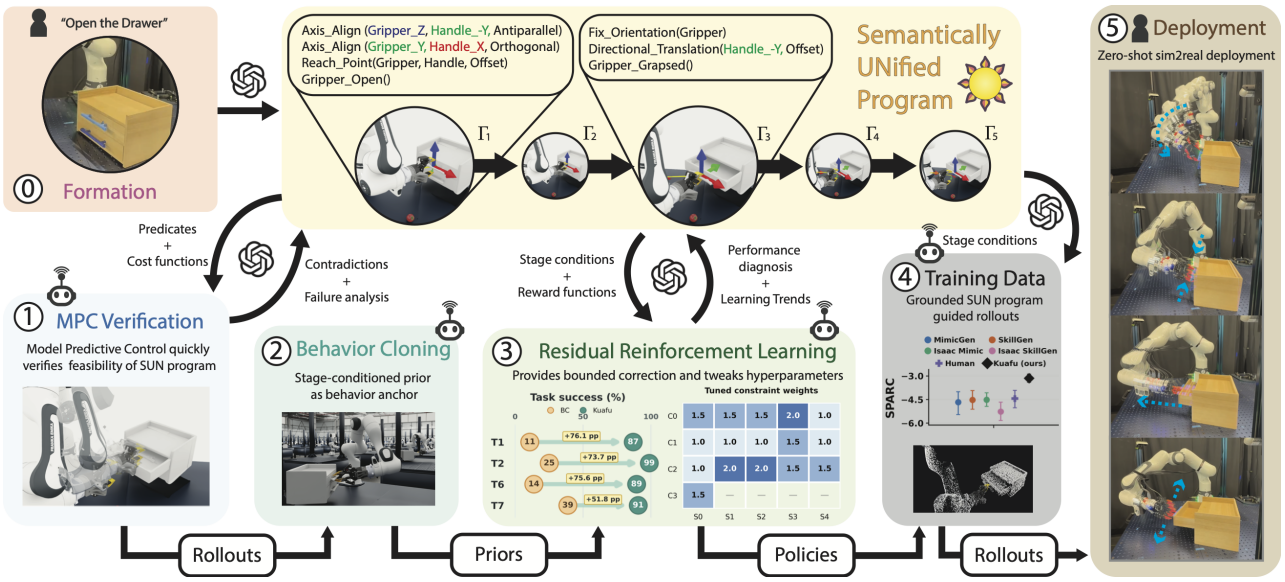


Fig. 2. The Kuafu harness for agentic robot policy learning. After video-based scene preparation, the agent forms and repairs a SUN Program (0), invokes MPC verification (1), and uses returned physical diagnostics to revise failed specifications. An accepted program supplies aligned control, monitoring, and learning interfaces for demonstration generation, Stage-BC (2), bounded residual RL (3), and evaluation/data production (4). Training feedback supports scalar calibration while accepted task semantics remain fixed. Forward arrows carry programs, policies, or data; return arrows carry diagnostics to the agent. The final handoff exports recorded observations and actions alone to a visual policy (5). Controllers execute the individual control steps.

duction around that same program, with diagnostic repair and constrained training calibration.

Across nine multi-stage tasks, we demonstrate that the harness and its ablations establish the complementary roles of program-derived supervision, the behavioral prior, and bounded correction. Our generated data support visual policies and vision-language-action (VLA) models, with zero-shot DP3 deployment on physical Franka and Kinova robots.

II. Related Work

Executable Task Representations. Multi-stage manipulation requires representations that bridge discrete task progression with continuous motion. Symbolic planning couples task structure to geometric feasibility [7]. Language-grounded approaches express tasks as spatial objectives, including composable 3D value maps, reward parameters evaluated by MPC [9, 10], and grounded keypoint constraints coupled with closed-loop optimization [11]. Formal methods encode temporal progress via finite-state rewards or signal-temporal-logic constraints for learning and predictive control [12, 13]. Unlike these works, SUN Programs do not reconstruct task semantics at each tool interface. Instead, they retain a physically tested program across control, monitoring, and learning modules, thereby preserving semantic consistency throughout the pipeline.

Control-to-Policy Transfer. Task structure enters policy learning through generated objectives or controller-derived supervision. Prior work synthesizes executable rewards and domain-randomization configurations directly from task descriptions [14–16]. However, iteratively refined objectives risk overfitting observed behavior or misstating the underlying task, meaning high return need not imply success on the intended physical outcome [17]. Alternatively, controller-derived supervision facilitates transfer via optimized trajectory targets [18], MPC teachers adapted to the learner's state distribution [19], and Hamiltonian losses encoding control constraints [20], while residual learning preserves nominal behavior by acquiring corrective actions [21]. Unlike these disjoint approaches, the Kuafu harness integrates these capabilities around a persistent SUN Program. Control verifies the candidate specification, demonstrations initialize the behavioral prior, and residual learning refines execution using the compiled interface—with the agent calibrating permitted scalars while the accepted task semantics remain fixed.

Agentic Robotics. LLM and VLM agents coordinate robotic planning, execution, and learning through generated programs and environmental feedback. Existing approaches ground action programs in robot capabilities and scene objects [3], incorporate environmental feedback into planning and execution [22, 23], and leverage generative agents to automate task construction and produce supervision for skill learning [2]. More recently, ASPIRE [24] distills execution-validated program repairs into reusable skills, while EMPIRE [25] supports real-world policy improvement through automated reset, verification, and agent-driven experimentation. Kuafu handles control-to-learning transfer by retaining the accepted SUN Program across all tools. This persistence allows policy learning to inherit physically tested task semantics directly, circumventing the need for the agent to repeatedly reconstruct objectives and completion conditions.

Autonomous Robot Data Generation. Autonomous generation integrates multimodal task creation with planning and RL solvers to synthesize demonstrations for downstream learning [26]. Other methods expand existing demonstrations via object-relative segment retargeting or planned connections between reusable skills [27, 28]. Unlike these approaches, Kuafu generates high-fidelity data by first verifying task specifications with MPC and then scaling production

via a learned controller. The retained SUN monitor supplies stage context, while an independent task-success criterion filters demonstrations, ensuring downstream policies learn only from physically valid trajectories.

III. Methods

Kuafu is an agent harness that converts language-described manipulation tasks into learned multi-stage robot policies. Given a natural-language instruction $\mathcal{I}$ and videos of the task objects, Kuafu produces a privileged simulation policy for scalable data generation and a visual policy for physical deployment. As summarized in Figure 2, the agent operates at task-level decision points: it forms and repairs SUN Programs using feedback from MPC verification, initiates behavioral-prior learning from successful rollouts, refines residual learning through multimodal reflection, and leverages the accepted program to guide data-generation rollouts. Crucially, the same accepted SUN Program persists from verification through policy learning and data production, ensuring the task is never reinterpreted at each tool handoff. Algorithm 1 formalizes this feedback-driven control flow, with superscripted step numbers (e.g., "$^{(2)}$Tool") corresponding to the workflow stages in Figure 2.

A. Scene Preparation and Agent Interface

Kuafu provides the agent with a robot model, simulation configuration, and a typed operator library $\mathcal{L}$. From the input videos, the agent invokes reconstruction tools to generate simulation-ready object assets, which are imported into the scene and grounded with semantic frames $\mathcal{F}$. These frames expose task-relevant object poses, axes, and reference points for binding to operators in $\mathcal{L}$. Together, $\mathcal{I}$, $\mathcal{F}$, and $\mathcal{L}$ constitute the language instruction, scene grounding, and executable vocabulary required to construct a SUN Program. Crucially, Kuafu instantiates an independent rule-based task-success criterion from the instruction prior to SUN Program synthesis; this evaluator verifies outcomes during program validation, policy learning, and data collection, ensuring ground-truth assessment remains decoupled from the generated programs.

We use "open the drawer" as a running example throughout this section. Here, the scene frames $\mathcal{F}$ identify the drawer and handle, defining their semantic axes (e.g., the handle-bar x-axis and drawer y-axis, with $+y$ defined as the closing direction). The operator library $\mathcal{L}$ provides typed relations and motion primitives such as `align_axis`, `direction_move`, and `fix_orientation`. The agent binds these operators to the relevant frames in $\mathcal{F}$ and sequences them into the ordered stages of a SUN Program.

B. SUN Programs as a Shared Task Representation

A SUN Program $\Gamma = (\Gamma_1, \ldots, \Gamma_K)$ serves as the executable task representation shared across Kuafu's control and learning tools. The agent constructs Γ by grounding operators from $\mathcal{L}$ in the semantic frames $\mathcal{F}$ and organizing the resulting relations into K ordered stages. For the "open the drawer" example, these relations specify conditions such as aligning the gripper with the handle, reaching the target, maintaining orientation, and translating the drawer along its opening axis. Each relation retains its grounded arguments, satisfaction boundary, and stage membership throughout the entire workflow. For every relation j in stage $k \in \{1, \ldots, K\}$, the SUN compiler exposes three executable views:

$$\mathrm{Compile}(\Gamma_k) = \left\{ \left(\phi_{k,j}^{\mathrm{mpc}}, P_{k,j}, c_{k,j}^{\mathrm{rl}} \right) \right\}_j, \tag{1}$$

where $\phi_{k,j}^{\mathrm{mpc}}$ is an MPC objective term, $P_{k,j}$ is a satisfaction predicate, and $c_{k,j}^{\mathrm{rl}}$ is an RL learning cost. These views are operator-specific and need not share the same numerical form; for instance, a point-reaching relation may employ a quadratic distance penalty for MPC, a tolerance test for satisfaction, and a bounded distance cost for learning. What remains invariant across these views is the relation itself: its grounding, satisfaction boundary, and position in the task sequence. We refer to this correspondence across control, monitoring, and learning as *Semantic Unification*.

The SUN Program also governs task progression. A runtime monitor maintains the active stage z_t at time t and stores an entry-state snapshot ξ_{z_t} containing the reference poses captured at the start of the current stage. After each action, it evaluates the stage's satisfaction predicates on the updated state, advancing only when all conditions are met. This identical stage logic is applied uniformly across MPC verification, Stage-BC, residual learning, and data production. Thus, downstream tools do not reconstruct task progression from raw trajectories or independently redefine completion conditions; they operate on the same verified task semantics established at the outset.

C. Verification and Program Repair

Kuafu verifies a candidate SUN Program via closed-loop MPC prior to policy learning (Step 1 in Figure 2). For each active stage, the verification tool optimizes the compiled MPC objectives, executes the resulting action, observes the new state, and re-evaluates the SUN satisfaction predicates before re-planning. In parallel, an independent rule-based

Algorithm 1 Kuafu agent harness with the SUN program.

Input: Instruction $\mathcal{I}$, object videos, operator library $\mathcal{L}$
$^{(0)}$**Tool:** reconstruct scene and expose frames $\mathcal{F}$
$^{(0)}$**Agent:** form SUN Program Γ from $(\mathcal{I}, \mathcal{F}, \mathcal{L})$
while verification budget remains **do**
  $^{(1)}$**Tool:** compile Γ; MPC verify on initial states
  if accepted then $\Gamma^\star \leftarrow \Gamma$; **break**
  $^{(1)}$**Agent:** repair Γ from feedback
end while
if no $\Gamma^\star$ **then return** formation not accepted
$^{(2)}$**Tool:** MPC rollouts $\rightarrow$ frozen Stage-BC π_θ
$^{(3)}$**Tool:** initialize residual learning using π_θ and $\Gamma^\star$
repeat
  $^{(3)}$**Agent:** propose weights w, scales α
  $^{(3)}$**Tool:** train/evaluate; **stop early if** $\mathrm{SR}_t < \beta\,\mathrm{SR}_{\max}$
until training completes without early stopping
$^{(4)}$**Tool:** roll out privileged policy π_{pri}: π_{pri} outputs actions; $\Gamma^\star$ advances and labels stages; retain successful trajectories
$^{(5)}$**Tool:** train and evaluate visual policy π_{vis} from the retained sensor–action trajectories
return Privileged policy π_{pri}, visual policy π_{vis}

task-success criterion evaluates whether the intended physical outcome has been achieved. Crucially, this check is generated independently of the candidate SUN Program and frozen before synthesis begins. For the "open the drawer" example, it declares success only when the drawer reaches the required opening displacement.

The resulting rollouts test two critical aspects: whether the grounded objectives are physically executable and whether the program's completion logic aligns with the intended task outcome. The tool returns detailed diagnostics identifying unsatisfied relations, measured constraint violations, stage failures, and *false completions*—cases where the SUN Program declares success, but the independent task-success check rejects the outcome. The agent uses this feedback to repair the earliest diagnosed failure while preserving all previously validated requirements, then resubmits the revised program for verification.

A candidate program is accepted if at least one rollout achieves the independently verified task outcome and no trajectory in the verification batch produces a *false completion*. Upon acceptance, the grounded relations, satisfaction boundaries, predicates, operator definitions, and stage order are frozen in $\Gamma^\star$. The agent ceases to revise the task semantics; subsequent interactions focus exclusively on learning how to execute the accepted program more robustly.

D. Policy Learning and Agent Calibration

Once a SUN Program $\Gamma^\star$ is accepted, Kuafu leverages its successful MPC rollouts to learn a stage-conditioned behavioral prior $\pi_\theta(o_t, z_t)$ (Step 2 in Figure 2). Here the observation o_t comprises privileged robot and object observations, while the active stage z_t is provided by the verified SUN monitor. The action u_t corresponds to a task-space end-effector target paired with a binary gripper command. Crucially, this prior learns how to execute the demonstrated behavior without needing to reconstruct task progression from raw trajectories. At execution time, Stage-BC replaces the computationally expensive online MPC optimization with fast, learned action prediction, while $\Gamma^\star$ retains sole authority over determining when to advance stages [29]. For the "open the drawer" example, o_t includes the end-effector, drawer, and handle poses, along with the gripper state.

Kuafu then freezes π_θ and learns a bounded residual policy π_ψ to refine execution (Step 3 in Figure 2). Following residual RL [21], the privileged whole-task policy is

$$\begin{aligned}\pi_{\mathrm{pri}}(o_t, z_t) &= \mathrm{clip}\big(\pi_\theta(o_t, z_t) + \alpha\pi_\psi(o_t, z_t), u_{\min}, u_{\max}\big),\\ u_t &= \pi_{\mathrm{pri}}(o_t, z_t), \qquad \|\pi_\psi(o_t, z_t)\|_\infty \le \rho_{\max}.\end{aligned} \tag{2}$$

Here, α scales the correction magnitude, $\rho_{\max}$ bounds each residual component, and $u_{\min}$, $u_{\max}$ denote environment action limits. Note that this learned action residual modifies control commands; it is distinct from the geometric residuals within SUN operators, which measure task satisfaction rather than modifying actions.

Residual learning inherits the learning costs compiled from $\Gamma^\star$, **avoiding the need to synthesize new task rewards**, ensuring stable behavior. For relation j in stage k, these costs $c^{\mathrm{rl}}_{k,j}(t)$ and the relative weights $w_{k,j}$ define the normalized progress score

$$S_k(t) = 1 - \frac{\sum_j w_{k,j} c^{\mathrm{rl}}_{k,j}(t)}{\sum_j w_{k,j}}, \tag{3}$$

where costs are evaluated against the current state and the stage-specific reference information stored by the SUN monitor. Higher values of $S_k(t)$ indicate greater progress toward the accepted stage requirements. The reward function then structures learning around incremental improvement beyond the best score previously attained in the active stage:

$$\begin{aligned}r_t = \lambda_{\mathrm{prog}}\left[S_{z_t}(t+1) - S^{\mathrm{best}}_{z_t}(t)\right]_+ + \lambda_{\mathrm{stage}}\mathbf{1}[z_{t+1} > z_t]\\ + \lambda_{\mathrm{true}}\mathbf{1}[\mathrm{TS}_t] - \lambda_{\mathrm{false}}\mathbf{1}[\mathrm{FC}_t] - \lambda_{\mathrm{act}}\|u_t\|_2^2.\end{aligned} \tag{4}$$

Here, TS_t and FC_t denote task-success and false-completion events, respectively. The term S^{best} initializes to zero upon stage entry, tracks the largest score achieved within that stage, and resets when the SUN monitor advances. Crucially, the progress term is evaluated prior to the stage transition, ensuring that both dense progress signals and stage advancement are measured against the same verified stage definition. Thus, the SUN Program not only dictates task progression but also defines the metric by which residual learning measures progress within each stage.

During the learning phase (lines 11–14 in Algorithm 1), the agent uses learning trends and stage-wise constraint satisfaction to calibrate the relative term weights $w_{k,j}$, residual strength α, and bound $\rho_{\max}$ (Step 3 in Figure 2). Each residual-RL iteration yields a training success rate SR_t. The process terminates early if SR_t drops below a fraction β of the peak success rate achieved so far; in this case, the agent updates the hyperparameters and re-trains. These updates are strictly limited to adjusting reward emphasis and the permitted deviation from the frozen Stage-BC prior. The agent is prohibited from altering the task semantics verified by the MPC mentor.

We define the sequence from an accepted SUN Program through its MPC mentor rollouts, Stage-BC prior, residual refinement, and final privileged policy as a *lineage*. Consequently, independent lineages jointly vary the program formation process, mentor trajectory data, and subsequent policy learning parameters, ensuring that each lineage represents a distinct, end-to-end execution of the full pipeline.

E. SUN-Guided Data Production and Deployment

After learning, Kuafu freezes the final privileged policy π_{pri} and validates it in simulation. The frozen controller then drives scalable data production (Step 4 in Figure 2). During each rollout, π_{pri} generates robot actions while the accepted SUN Program $\Gamma^\star$ remains active as the task monitor. It supplies the current stage z_t to the Stage-BC and residual policies, advances the task using the predicates verified by MPC, and records corresponding stage labels. Only trajectories satisfying the independent task-success check are retained. Thus, the learned policy executes the

TABLE I

FORMATION, VERIFICATION, AND AGENT INTERACTION COST.

Metric / method	T0	T1	T2	T3	T4	T5	T6	T7	T8	Mean
A. SUN Program formation and refinement: accepted programs and formation cost										
Stages K	3	8	6	6	7	9	15	14	10	8.67
Constraints	12	25	19	19	27	31	53	44	31	29.00
Formation iter.	1	1	1	1	2	2	2	1	1	1.33
Refinement iter.	0	0	0	0	1	1	0	0	2	0.44
Agent calls/task	11.2	19.8	15.2	16.2	24.6	29.2	37.0	32.8	28.8	23.87
B. Five repeated formation-and-MPC-verification runs per task										
Success w/o rec.	5/5	5/5	5/5	4/5	0/5	1/5	5/5	2/5	2/5	29/45
Recovery entry	0/5	0/5	0/5	1/5	5/5	4/5	0/5	3/5	3/5	16/45
Final success	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	**3/5**	**5/5**	**43/45**
Recovery cycles	0	0	0	2	5	9	0	11	3	30
C. Comparator agent interaction cost (tokens in thousands)										
Kuafu tokens/task	37.6	84.0	57.1	62.6	110.1	144.1	183.7	152.5	163.1	110.50
Eureka tokens/task	224.3	246.2	241.1	227.2	208.7	243.0	208.9	248.8	261.0	234.40
VoxPoser calls/run	12.9	13.9	13.0	13.0	16.1	21.8	18.8	19.9	23.5	16.99
VoxPoser tokens/run	25.1	24.6	24.9	23.6	27.2	38.0	31.4	39.7	49.8	31.60

motion, while SUN organizes the task logic and annotates the resulting demonstrations. The retained rollouts provide the observation-action pairs used to train the downstream visual policy π_{vis} (Step 5 in Figure 2), which is subsequently deployed on the physical robot.

IV. EXPERIMENTS

We address four central questions in our experiments: (1) *Formation Reliability:* How reliably can SUN Programs be synthesized from language and scene semantics? (2) *Performance Elevation:* Does the persistent support of SUN Programs significantly elevate task success compared to non-persistent baselines? (3) *Data Efficiency:* Can Kuafu outperform existing data-generation methods in terms of trajectory quality and sample efficiency? (4) *Visual Policy:* Does the dataset produced by Kuafu enable visual policies and robust zero-shot transfer to physical robots? As shown in Figure 1, we designed nine multi-stage tasks around constrained motion, precise placements, orientation-sensitive grasping, and high-precision manipulation to evaluate physical capabilities that recur throughout everyday manipulation.

E-stop interactions (T0: push, T3: twist), *stacking* (T1: two cubes, T6: three cubes), *drawer manipulation* (T2: pull open, T7: open and place object), *object reorientation* (T4: right a bottle), *insertion* (T5: marker into cup), and *dexterous hanging* (T8: hang cup by handle). The tasks isolate different forms of compositional scalability for objects and interactions. T0/T3 vary the complexity of contact-constrained motion on the same mechanism. T6 extends T1 by repeating its pick-and-place sequence. T7 combines T2's drawer motion with T1's pick-and-place skill. T4/T5 both require object reorientation, but T5 requires an additional inter-object geometric constraint. Lastly, T5 and T8 compare high-precision manipulation across distinct object morphologies and contact geometries.

We report overall performance as task-macro success: the unweighted mean of the success rate across all tasks. The agents in the primary Kuafu, Eureka, and VoxPoser experiments use the same underlying model, GPT-5.5, for all task-level reasoning calls. All methods share the same task-specific initial-condition sampler.

A. Formation Reliability

Panels A and B of Table I demonstrate results for program formation and refinement. Of 45 independent runs, 29 passed initially; diagnostic repair recovered 14 of the remaining 16, raising the yield from 64.4% to 95.6%. Recovery was most critical for T4 and T5, improving acceptance from 10% to 100%. T7 was the only task with 2 unrecovered runs, and it accounted for 11 of the 30 total recovery cycles. Since the larger 15-stage T6 program succeeded in all five runs without recovery, raw program size alone does not explain formation difficulty. The most common errors in parsed programs involve axis-reference signs (i.e., mistaking the direction of operation) and axis relations (i.e., confusing orthogonal and parallel relations), suggesting that the agent struggles with spatial reasoning, especially for orientation-heavy tasks. Since T7 combines constrained movement with a tight orientation requirement and a separate pick-and-place action, the unrecovered runs suggest that combining heterogeneous manipulation phases poses a greater challenge. These results establish diagnostic-guided repair as an effective mechanism for recovering initially invalid programs.

Panel C reports costs of the agentic harness. Across the 9 tasks, Kuafu consistently used fewer tokens than Eureka, which rewrites the reward functions in each iteration. At the reported mean rates, VoxPoser's cumulative costs will exceed Kuafu's after just two executions (calls) and four rollouts (tokens). These results demonstrate the benefits of forming a persistent program, which converts a recurring agent interaction cost into a task-level cost.

B. Performance and Ablation Analysis

Panel A of Table II examines the control-to-learning transfer through component ablations and alternative learners, using MPC's 83.76% task-macro success as the reference. Pure RL is effectively unsuccessful with either the naive or SUN reward. With the SUN reward, removing the accepted Stage-BC prior leaves learning trapped in unsuccessful early-stage behaviors. When constraints are active, the policy struggles to balance geometric terms and often prioritizes their orientation constraints while moving away from positional targets, eventually reaching an equilibrium in which exploration stalls. Although each reward term is numerically bounded and verified, individual terms—or groups of simultaneously active terms—can still dominate the learned behavior. Thus, program-derived rewards alone do not enable the learner to discover useful behavior.

Demonstration supervision provides a starting point, but its effectiveness depends on how task progress is represented. Stage-free BC leaves several tasks unsolved, whereas Stage-BC achieves nonzero success throughout the suite. This suggests that explicit progress context helps distinguish the control decisions at different stages. Nevertheless, the remaining gap from MPC shows that imitation alone does not recover the mentor's performance. Residual learning does not consistently repair an incomplete handoff. Starting from stage-free BC, residual learning improves selected tasks

TABLE II

CONTROL-TO-LEARNING HANDOFF AND BASELINES

Variant / method	T0	T1	T2	T3	T4	T5	T6	T7	T8	Mean
*A. **Kuafu** component-ablation success (%): partial handoffs and alternative state-policy learners*										
MPC	100.00	96.15	100.00	89.29	62.50	58.14	94.34	70.92	82.50	83.76
Pure RL, naive reward	0.04	0.00	0.13	0.00	0.43	0.00	0.00	0.00	0.00	0.07
Pure RL, SUN reward	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Stage-free-BC	52.20	12.16	3.38	11.17	5.29	0.00	0.00	0.00	0.77	9.44
Stage-BC	36.01	10.79	25.11	29.80	22.47	31.82	13.81	38.80	14.16	24.75
Stage-free-BC + RRL	1.20	67.30	0.03	0.00	60.31	17.49	3.83	0.16	16.69	18.56
Bounded RRL + Sparse Rewards	58.67	87.20	49.56	58.67	58.96	0.00	0.00	7.95	0.00	35.67
TD-MPC2 [30], naive reward	79.64	27.00	0.00	5.99	1.07	0.00	0.00	0.00	0.00	12.63
TD-MPC2 + prior, naive reward	77.58	13.00	97.47	**99.27**	0.20	0.00	0.00	0.00	0.00	31.95
Kuafu (Primary)	**98.48**	**95.78**	**98.01**	84.62	**58.80**	**69.70**	**87.25**	**95.21**	**50.43**	**82.03**
Kuafu (3 Runs)	98.37±0.81	94.75±0.62	97.58±1.04	82.09±0.65	56.24±1.81	62.04±5.34	84.72±1.32	94.06±1.06	44.99±1.84	79.43
B. Baseline performance success (%): iterative reward search (Eureka) and online planning (VoxPoser)										
Eureka: Best Train ≤ 5	60.53	0.02	78.96	18.17	6.47	0.01	0.00	0.01	0.01	18.24
Eureka: Final Eval.	25.85	0.01	71.75	15.16	2.83	0.01	0.00	0.00	0.01	12.85
VoxPoser: Direct	50.00	10.00	0.00	20.00	0.00	0.00	10.00	0.00	0.00	10.00

but severely degrades T0, T2, and T3. Retaining the stage-conditioned prior improves aggregate performance further under sparse rewards, yet T5, T6, and T8 become entirely unsuccessful. These regressions suggest that sparse feedback provides insufficient guidance to preserve and refine intermediate behavior. A useful initialization and action corrections therefore do not, by themselves, ensure successful refinement, especially for longer tasks. The alternative learner exhibits a similarly selective benefit from demonstrations. TD-MPC2 with the naive reward performs best on T0; adding a behavioral prior produces strong results on T2 and T3 but leaves T5–T8 unsolved. Its strongest performance remains concentrated on the shorter tasks rather than extending throughout the suite. Under this reward, a behavioral prior helps the learner acquire selected tasks but is insufficient to reproduce the coverage of the complete handoff.

Combining the stage-conditioned prior, program-derived supervision, and bounded correction, Kuafu reaches 82.03% in the primary experiment, the highest task-macro success among the evaluated learned variants in Panel A. Unlike the partial handoffs, it maintains success throughout the task suite. The comparison with sparse-reward refinement is particularly informative: with the Stage-BC prior and residual bound unchanged, SUN supervision recovers successful completion on every task left unsolved by that variant. Together with the pure-RL failures, this supports complementary roles for the prior and program-derived supervision: the prior supplies useful behavior that reward-driven exploration fails to discover, while the program guides its refinement. The three-run results for Kuafu retain broad task coverage when the accepted program, MPC mentor set, and residual training vary jointly. This supports the repeatability of the complete pipeline beyond a single lineage. The paired tasks show Kuafu's robustness to longer stage sequences. Across the stacking and drawer pairs (T1/T6, T2/T7), Kuafu has much smaller success-rate gaps between the shorter and longer variants than the sparse-reward RRL ablation. More broadly, no non-Kuafu learned handoff or baseline reaches 50% on either T6 or T7, with taskwise maxima of only 13.81% and 38.80%, whereas Kuafu reaches 87.25% and 95.21%. This shows that the retained SUN Program preserves competence as stage sequences lengthen rather than merely improving isolated tasks.

TABLE III

CONTROLLER-DEVELOPMENT COMPUTE (GPU-HRS)

	Kuafu					Eureka		
	Form.	MPC	BC	RRL	Total	Search	Final	Total
Sum	1.55	69.03	3.21	48.86	**122.67**	412.24	67.36	**479.60**

Baseline Analysis. Conversely, baselines that rely on iterative reward search or online planning without persistent task semantics struggle to achieve robust, multi-stage success (Panel B). Eureka exhibits larger gaps between training and evaluation, indicating reward overfitting. Its failed controllers exhibit the same early fixation on selected geometric features observed in the SUN-reward runs without a prior. This shared failure suggests that revising rewards alone cannot navigate through multiple complex objectives. VoxPoser achieves only 10.00% success, completely failing on five tasks. Kuafu's gains stem from integration of a verified prior, program-derived supervision, and bounded correction, which keep the task specification aligned across control and learning.

Table III reports controller-development compute on the prepared simulation scenes, covering program formation, MPC verification, mentor trajectory collection, Stage-BC, and bounded residual learning. Kuafu's entries include unsuccessful loops during formation and calibrations. Summation is across all 9 tasks, under single-GPU worker-hour accounting. Eureka consumes 3.91 times Kuafu's compute. Kuafu's cost is dominated by MPC mentor-trajectory collection and residual RL, whereas SUN Program formation and validation require only 0.172 hours per task. This small overhead demonstrates the computational benefit of an executable task representation: candidate SUN programs can be evaluated directly through control without training a policy for every candidate. Thus, Kuafu's superior performance is achieved at a fraction of the computational cost, ruling out compute budget as a confounding factor.

C. Data Generation Efficiency and Data Quality

During data collection, the learned controller replaces online MPC, while the retained SUN Program continues to supply stage annotations. Although the MPC mentor leads in overall success rates (as shown in Panel A of Table II), its

TABLE IV

KUAFU DATA GENERATION

A. Successful trajectory-time throughput (min/active production hr, ↑)

Task	Human	MimicGen	SkillGen	Isaac Lab Mimic	Isaac Skill Mimic	**Kuafu**
Task macro	**23.34**	**29.36**	**26.48**	**32.19**	**24.79**	**246.79**
vs. Human	1.00	1.26	1.13	1.38	1.06	**10.57**

B. Controller readiness and break-even

	Controller ready	Time break-even	Cost break-even
Task macro	**13.63 GPU h** **$6.95**	**513 demos** **29.10 min**	**35.22 demos** **1.99 min**
Across-task range	4.84–25.52 GPU h $2.47–$13.02	225–854 demos 4.35–76.19 min	16–57 demos 0.30–5.09 min

C. MPC-relative Q95 yield and successful-trajectory diagnostics

Method	Q95 (%↑)	Time/ MPC_{50} ↓	EEF path/ MPC_{50} ↓	Action path/ MPC_{50} ↓	Action Δ^2/ MPC_{50} ↓
MimicGen	20.4	1.11	1.14	1.94	1.27
SkillGen	6.9	1.66	1.15	3.01	1.28
Isaac Lab Mimic	12.1	1.12	1.38	1.96	1.25
Isaac SkillGen	6.2	1.61	1.46	2.89	1.21
Kuafu (ours)	**57.3**	**1.04**	**0.67**	**0.52**	**0.28**
Human reference	0.0	3.06	1.59	2.81	1.25

D. Downstream DP3 success (%) (500 successful trajectories per task)

Method	T0	T1	T2	T3	T4	T5	T6	T7	T8	Mean
MimicGen	94.52	4.95	36.12	33.33	8.27	0.00	0.00	0.00	0.00	19.69
SkillGen	**97.07**	3.91	30.65	62.00	8.05	0.00	0.00	0.00	0.26	22.44
IL Mimic	90.78	14.21	40.92	36.54	**9.35**	0.00	0.00	0.00	0.00	21.31
IS Mimic	72.82	1.04	22.54	15.25	5.47	0.00	0.00	0.00	0.26	13.04
Kuafu	95.70	**58.70**	**55.50**	**85.30**	0.78	**15.80**	**12.70**	**69.00**	**20.70**	**46.02**

online optimization loop and single-environment execution limit demonstration throughput severely. We therefore exclude MPC from the data-collection throughput comparison. Table IV shows the evaluation results of the data engine through production throughput and controller amortization, MPC-relative trajectory diagnostics, and downstream visual-policy learning under a matched successful-data budget.

We first assess production efficiency, accounting for both collection throughput and the initial time and cost needed to prepare the controller (Panels A and B). Human baselines were collected via simulation teleoperation using an AgileX PIKA gripper, with measured poses defining the simulated end-effector targets. Once the final state controller is ready, Kuafu produces 10.57× as much successful trajectory time per active production hour as human teleoperation. To contextualize, MimicGen's data-generation experiments use 1,000 successful demonstrations per task variant [27]. Including controller preparation, Kuafu catches up with human teleoperation in cumulative successful demonstration count before reaching 1000 demonstrations on every task. Pricing human teleoperation at the U.S. federal minimum wage of $7.25/h and A6000 rental for both controller preparation and data production at $0.51/GPU h, Kuafu reaches cost parity after 35 successful demonstrations per task on average.

We next assess trajectory efficiency and smoothness relative to the accepted MPC mentor (Panel C). Q95 yield is the fraction of evaluated rollouts that succeed and whose completion time, end-effector path length, action path length, and action second difference are each no greater than the corresponding task-specific 95th percentile among successful MPC trajectories. The four diagnostics are normalized by task-specific medians from successful MPC trajectories (MPC_{50}). Kuafu achieves the highest Q95 yield, while its successful rollouts combine near-MPC completion times with shorter end-effector paths and lower action variation than competing generators. Additionally, we measure motion smoothness using the spectral arc length (SPARC) of measured end-effector translational speed at Step 4 of Figure 2, where higher values indicate smoother motion [31]. Across all nine tasks, Kuafu achieves a task-balanced SPARC of -3.14, compared with -4.44 for human demonstrations, -4.52 for SkillGen, and -4.67 for MimicGen. Results show that smoothness improvement can be seen in both manipulator's motion and action sequences.

TABLE V

VISUAL POLICY PERFORMANCE AND ZERO-SHOT TRANSFER

A. Single-task visual-policy success (%; same Kuafu data)

RGB	ACT [32] 35.40	Diffusion Policy **38.52**	smolVLA [33] 26.73	ManiFlow-RGB [34] 34.09
Point cloud	DP3 46.02	FlowPolicy [35] 21.01	ManiFlow-PC [34] **46.46**	MP1 [36] 12.94

B. Per-task maxima and state-controller success (%)

	T0	T1	T2	T3	T4	T5	T6	T7	T8	Mean
RGB max	84.53	30.54	73.98	60.70	**25.50**	10.10	12.10	**73.00**	**23.00**	43.72
PC max	**98.38**	**65.53**	**86.28**	**85.30**	10.05	**15.80**	**12.70**	69.00	20.70	**51.53**
State	98.48	95.78	98.01	84.62	58.80	69.70	87.25	95.21	50.43	82.03

C. Multi-task VLA success (%; one policy)

Policy	T0	T1	T2	T3	T4	T5	T6	T7	T8	Mean
SmolVLA [33]	5.87	0.26	12.94	2.08	0.00	0.00	0.00	0.00	0.00	2.35
WALL-X [37]	22.22	**1.56**	60.07	**11.78**	0.00	0.00	0.00	10.51	0.00	11.79
$\pi_{0.5}$ [38]	**26.61**	0.00	**81.87**	0.00	4.17	4.17	0.00	**69.17**	0.00	**20.67**

Efficiency and smoothness alone do not show whether the data support a visual learner, so we test utility directly in downstream policy learning (Panel D). With a fixed 3D Diffusion Policy (DP3) learner and a matched budget of 500 successful trajectories per task, Kuafu yields data that improve visual-policy success by 23.6 percentage points over SkillGen, the strongest alternative. The improvement also can be seen from task coverage: Kuafu alone yields nonzero downstream success on all nine tasks, including T5–T7, where every competing source yields none. Kuafu therefore not only generates demonstrations faster than competing methods, but also produces more effective training data.

D. Visual Learning and Zero-Shot Transfer

Table V presents the utility of Kuafu's trajectories after removing privileged state and stage inputs. For single-task policies (Panel A), ManiFlow-PC preserves fine-grained point features and uses adaptive cross-attention for flow-based action generation, while DP3 conditions diffusion-based action sequences on a compact 3D representation. Their gains over their RGB counterparts and other point-cloud methods suggest that explicitly encoding spatial relationships helps reproduce precise manipulation behaviors without recovering those relationships from images and point clouds alone.

Panel B reports per-task maxima within each modality family; these represent optimal architectures for each task, not a single deployable oracle. While maxima nearly match the state controller on T0 and exceed it on T3, the 15-stage T6 shows the largest gap of 74.55 points. These gaps, along with the multi-task results, reflect losses from observation conditioning, partial observability, and architectural constraints rather than state-based execution alone. With one policy across all nine tasks, $\pi_{0.5}$ reaches 20.67% task-macro success, much higher than WALL-X and SmolVLA (Panel C). SmolVLA prioritizes a compact, efficient architecture, WALL-X emphasizes tightly coupled language–action learning, and $\pi_{0.5}$ emphasizes transfer from heterogeneous robot and semantic pretraining. With identical demonstrations, SmolVLA struggles even on tasks that the other two models learn successfully. Although $\pi_{0.5}$ performs best overall, its advantage over WALL-X is concentrated in T2 and T7. This pattern is consistent with differences in model capacity and pretrained representations affecting how effectively each learner uses the same supervision. The shared failures further indicate that these learner-dependent gains improve selected behaviors, while the policies still struggle with longer tasks.

For physical deployment, we train DP3 on Kuafu demonstrations. We use 1,000 trajectories per task and deploy the resulting policies without real-world fine-tuning. Across six tasks and three robot–gripper configurations, DP3 achieves 34.0% success over 106 trials, with successful executions on every task. T4 also achieves a higher success rate in deployment than in simulation. These experiments demonstrate that Kuafu-generated demonstrations provide usable supervision for visual policies deployed on physical hardware, closing the loop from SUN Program supervision through simulated demonstration generation to real-robot execution.

V. Conclusions

We presented SUN Programs—persistent executable task artifacts that inject MPC-verified semantics into policy learning—and Kuafu, an agentic harness that builds robot policies around these artifacts. Kuafu achieves 82.03% task-macro success on the primary evaluation, compared with 35.67% for the matched sparse-reward baseline. Ablations confirm the complementary roles of program-derived supervision, the verified behavioral prior, and bounded residual correction. Furthermore, Kuafu-generated data yield 46.0% success in fixed-volume DP3 training, support 8 single-task visual policies and 3 multi-task VLAs, and enable zero-shot transfer across 106 physical trials. These results demonstrate how control-verified task knowledge can underpin both robust learned execution and observation-only visual policies.

Scope and limitations. The current harness relies on rich object semantics and a finite typed-operator library, with a monotone stage monitor that advances through ordered sequences of stages without recovery. Extending the operator vocabulary and supporting stage recovery are natural next steps. Additionally, our physical evaluation assumes well-calibrated real-world sensors matching simulation conditions. Currently, Kuafu does not handle moving obstacles or external disturbances during execution.

References

[1] J. Liang, W. Huang, F. Xia *et al.*, "Code as policies: Language model programs for embodied control," arXiv preprint arXiv:2209.07753, 2022.

[2] Y. Wang, Z. Xian, F. Chen *et al.*, "RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 235. PMLR, 2024, pp. 51 936–51 983.

[3] I. Singh, V. Blukis, A. Mousavian *et al.*, "ProgPrompt: Generating situated robot task plans using large language models," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 11 523–11 530.

[4] H. He, W. Yao, K. Ma *et al.*, "WebVoyager: Building an end-to-end web agent with large multimodal models," in *Proc. ACL Annual Meeting (Vol 1: Long Papers)*, 2024, pp. 6864–6890.

[5] Y. Shao, Y. Jiang, T. Kanell, P. Xu, O. Khattab, and M. Lam, "Assisting in writing Wikipedia-like articles from scratch with large language models," in *Proc. NAACL Conf.: Human Lang. Technol. (Vol. 1: Long Papers)*, 2024, pp. 6252–6278.

[6] C. Zhang, L. Li, S. He *et al.*, "UFO: A UI-focused agent for Windows OS interaction," in *Proc. NAACL Conf.: Human Lang. Technol. (Vol. 1: Long Papers)*, 2025, pp. 597–622.

[7] M. Toussaint, "Logic-geometric programming: An optimization-based approach to combined task and motion planning," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2015, pp. 1930–1936.

[8] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[9] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D value maps for robotic manipulation with language models," in *Proc. Conf. Robot Learn. (CoRL)*, vol. 229. PMLR, 2023, pp. 540–562.

[10] W. Yu, N. Gileadi, C. Fu *et al.*, "Language to rewards for robotic skill synthesis," in *Proc. Conf. Robot Learn. (CoRL)*, vol. 229. PMLR, 2023, pp. 374–404.

[11] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, "ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation," in *Proc. Conf. Robot Learn. (CoRL)*, vol. 270. PMLR, 2025, pp. 4573–4602.

[12] R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "Using reward machines for high-level task specification and decomposition in reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 80, 2018, pp. 2107–2116.

[13] Y. Meng and C. Fan, "Signal temporal logic neural predictive control," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7719–7726, 2023.

[14] T. Xie, S. Zhao, C. H. Wu *et al.*, "Text2Reward: Reward shaping with language models for reinforcement learning," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, spotlight.

[15] Y. J. Ma, W. Liang, G. Wang *et al.*, "Eureka: Human-level reward design via coding large language models," in *Proc. Int. Conf. Learn. Rep. (ICLR)*, 2024.

[16] Y. J. Ma, W. Liang, H.-J. Wang *et al.*, "DrEureka: Language model guided sim-to-real transfer," in *Robotics: Science and Systems*, 2024.

[17] S. Booth, W. B. Knox, J. Shah, S. Niekum, P. Stone, and A. Allievi, "The perils of trial-and-error reward design: Misdesign through overfitting and invalid task specifications," in *Proc. AAAI Conf. on AI*, vol. 37, no. 5, 2023, pp. 5920–5929.

[18] S. Levine and V. Koltun, "Guided policy search," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 28. PMLR, 2013, pp. 1–9.

[19] G. Kahn, T. Zhang, S. Levine, and P. Abbeel, "PLATO: Policy learning using adaptive trajectory optimization," in *2017 IEEE International Conference on Robotics and Automation*. IEEE, 2017, pp. 3342–3349.

[20] J. Carius, F. Farshidian, and M. Hutter, "MPC-Net: A first principles guided policy search," *IEEE Rob. and Auto. Let.*, vol. 5, no. 2, pp. 2897–2904, 2020.

[21] T. Johannink, S. Bahl, A. Nair *et al.*, "Residual reinforcement learning for robot control," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6023–6029.

[22] W. Huang, F. Xia, T. Xiao *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1769–1782.

[23] M. Pan, J. Zhang, T. Wu, Y. Zhao, W. Gao, and H. Dong, "OmniManip: Towards general robotic manipulation via object-centric interaction primitives as spatial constraints," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 17 359–17 369.

[24] R. Lu, Y. Wu, E. Kou *et al.*, "ASPIRE: Agentic /skills discovery for robotics," *arXiv preprint arXiv:2607.00272*, 2026.

[25] W. Xiao, J. Xie, T. Zhang *et al.*, "ENPIRE: Agentic robot policy self-improvement in the real world," arXiv preprint arXiv:2606.19980, 2026.

[26] P. Hua, M. Liu, A. Macaluso *et al.*, "GenSim2: Scaling robot data generation with multi-modal and reasoning llms," in *Proc. Conf. Robot Learn. (CoRL)*, vol. 270. PMLR, 2025, pp. 5030–5066.

[27] A. Mandlekar, S. Nasiriany, B. Wen *et al.*, "MimicGen: A data generation system for scalable robot learning using human demonstrations," in *Proc. Conf. Robot Learn. (CoRL)*, 2023.

[28] C. R. Garrett, A. Mandlekar, B. Wen, and D. Fox, "SkillMimicGen: Automated demonstration generation for efficient skill learning and deployment," in *Proc. Conf. Robot Learn. (CoRL)*, vol. 270. PMLR, 2025, pp. 2750–2790.

[29] A. Byravan *et al.*, "Evaluating model-based planning and planner amortization for continuous control," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[30] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[31] S. Balasubramanian, A. Melendez-Calderon, A. Roby-Brami, and E. Burdet, "On the analysis of movement smoothness," *J. Neuroeng. Rehabil.*, vol. 12, p. 112, 2015.

[32] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. Robot.: Sci. Syst. (RSS)*, Daegu, Republic of Korea, 2023.

[33] M. Shukor, D. Aubakirova, F. Capuano *et al.*, "SmolVLA: A vision-language-action model for affordable and efficient robotics," *arXiv:2506.01844*, 2025.

[34] G. Yan, J. Zhu, Y. Deng *et al.*, "ManiFlow: A general robot manipulation policy via consistency flow training," in *Proc. Conf. Robot Learn. (CoRL)*, vol. 305. PMLR, 2025, pp. 2268–2293.

[35] Q. Zhang, Z. Liu, H. Fan, G. Liu, B. Zeng, and S. Liu, "FlowPolicy: Enabling fast and robust 3D flow-based policy via consistency flow matching for robot manipulation," in *Proc. AAAI*, vol. 39, no. 14, 2025, pp. 14 754–14 762.

[36] J. Sheng, Z. Wang, P. Li, and M. Liu, "MP1: MeanFlow tames policy learning in 1-step for robotic manipulation," in *Proc. AAAI*, vol. 40, no. 22, 2026.

[37] A. Zhai, B. Liu, B. Fang *et al.*, "Igniting VLMs toward the embodied space," *arXiv preprint arXiv:2509.11766*, 2025.

[38] K. Black, N. Brown, J. Darpinian *et al.*, "$\pi_{0.5}$: A vision-language-action model with open-world generalization," in *Proc. Conf. Robot Learn. (CoRL)*, vol. 305. PMLR, 2025, pp. 17–40.

[39] B. Ichter, A. Brohan, Y. Chebotar *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *Proc. Conf. Robot Learn.*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 287–318.

[40] B. Cicek, M. K. Er, and O. S. Oguz, "CoRAL: Contact-rich adaptive LLM-based control for robotic manipulation," arXiv preprint arXiv:2605.02600, 2026.

[41] K. Torshizi, A. Singh, S. Mathur, K. Habib, L. Du, and P. Tokekar, "STeP: Signal temporal logic for precise specifications for action generation with vision language models," arXiv preprint arXiv:2607.18580, 2026.

[42] T. Mu, M. Liu, and H. Su, "DrS: Learning reusable dense rewards for multi-stage tasks," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[43] A. López Escoriza, N. Hansen, S. Tao, T. Mu, and H. Su, "Multi-stage manipulation with demonstration-augmented reward, policy, and world model learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.

[44] S. Patel, X. Yin, W. Huang *et al.*, "A real-to-sim-to-real approach to robotic manipulation with VLM-generated iterative keypoint rewards," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 8258–8266.

[45] J. Zhang, Y. Luo, A. Anwar *et al.*, "ReWiND: Language-guided rewards teach robot policies without new demonstrations," arXiv preprint arXiv:2505.10911, 2025.

[46] Q. Chen, H. Zheng, J. Yu *et al.*, "SARM2: Multi-task stage aware reward modeling for self improving robotic manipulation," arXiv preprint arXiv:2606.10305, 2026.

[47] R. Gong, X. Zhang, J. Shang *et al.*, "AnyTask: An automated task and data generation framework for advancing sim-to-real policy learning," arXiv preprint arXiv:2512.17853, 2025.

APPENDIX I
POSITIONING RELATIVE TO RELATED WORK

Table VI compares task representations and the capabilities they support in online execution, policy learning, and automated task-to-policy workflows. Its ten criteria cover explicit task specifications, stage structure, grounded geometry, control, verification, learning, and execution diagnostics. Each entry indicates demonstrated support in the specified role.

Representative systems	**Task spec.**	**Stage aware**	**Stage guards**	**Structured geometry**	**Direct control**	**Pre-policy verification**	**Policy learning**	**Whole-task policy**	**Spec. at execution**	**Execution diag.**
A. Language-generated task representations used for online execution										
SayCan [39]	✓	✓	✓	✗	✓	✗	✗	✗	✓	✗
Code as Policies [1]	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
ProgPrompt [3]	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Language to Rewards [10]	✓	✗	✗	✓	✓	✗	✗	✗	✗	✗
VoxPoser [9]	✓	✓	✗	✓	✓	✗	✗	✗	✗	✗
ReKep [11]	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
OmniManip [23]	✓	✓	✗	✓	✓	✗	✓	✓	✗	✗
CoRAL [40]	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
STeP [41]	✓	✓	✓	✓	✓	✗	✗	✗	✓	✓
B. Generated or learned task supervision used for policy learning										
Text2Reward [14]	✓	✓	✓	✗	✗	✗	✓	✗	✗	✗
Eureka / DrEureka [15, 16]	✓	✗	✗	✗	✗	✗	✓	✗	✗	✗
DrS [42]	✗	✓	✓	✗	✗	✗	✓	✓	✗	✗
DEMO3 [43]	✗	✓	✓	✗	✓	✗	✓	✓	✓	✗
IKER [44]	✓	✓	✓	✓	✗	✗	✓	✗	✓	✗
ReWiND [45]	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
SARM2 / SPIRAL [46]	✗	✓	✓	✗	✗	✗	✓	✓	✗	✗
C. Automated task-to-policy workflows										
AnyTask [47]	✓	✓	✗	✗	✓	✓	✓	✓	✗	✗
SUN Program / Kuafu (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

TABLE VI

TASK REPRESENTATIONS AND SUPPORTED CAPABILITIES. ✓ DENOTES EXPLICIT AND EXPERIMENTALLY DEMONSTRATED SUPPORT; × DENOTES SUPPORT NOT DEMONSTRATED IN THE SPECIFIED ROLE. **TASK SPEC.**: AN EXPLICIT, SEPARATELY REPRESENTED, INSPECTABLE TASK SPECIFICATION; **STAGE AWARE**: DECOMPOSES, SEQUENCES, LABELS, OR CONDITIONS SUPERVISION ON DISTINCT TASK STAGES; **STAGE GUARDS**: STATE-DEPENDENT CONDITIONS THAT GOVERN STAGE TRANSITIONS; **STRUCTURED GEOMETRY**: TASK-RELEVANT GEOMETRY REPRESENTED THROUGH RESIDUALS, ROBOT OR OBJECT FRAMES AND AXES, 3D KEYPOINT CONSTRAINTS, SPATIAL VALUE MAPS, CONTACT POINTS, NORMALS OR REGIONS, CLEARANCES, OR FORMAL METRIC PREDICATES; **DIRECT CONTROL**: THE TASK REPRESENTATION IS CONSUMED DIRECTLY BY A PLANNER OR CONTROLLER; **PRE-POLICY VERIFICATION**: A GENERATED TASK SPECIFICATION OR REWARD IS VERIFIED BEFORE POLICY LEARNING; **POLICY LEARNING**: THE TASK REPRESENTATION DIRECTLY SUPPLIES LEARNING SUPERVISION; **WHOLE-TASK POLICY**: ONE LEARNED CONTROLLER FOR THE ENTIRE MULTI-STAGE TASK; **SPEC. AT EXECUTION**: THE TASK REPRESENTATION REMAINS CONSULTED DURING EXECUTION; AND **EXECUTION DIAG.**: DIAGNOSTICS LOCALIZE TERM-LEVEL FAILURES DURING EXECUTION.

The SUN Program serves as the shared task contract in the Kuafu harness: physically verified relations remain available to guide policy learning and monitor execution. This continuity preserves accepted task semantics across the agent's tool calls.

APPENDIX II
ADDITIONAL SUN IMPLEMENTATION DETAILS

This appendix specifies the grounding conventions, compiled operator semantics, and policy interfaces used by the Kuafu harness. Each SUN relation provides the MPC objective term, satisfaction predicate, and RL learning cost introduced in Section III-B. The following sections describe how these interfaces share grounded arguments and stage-entry references, how the receiving application assigns tolerances, and how the learned policies represent actions.

We use the main text's notation: $k \in \{1, \ldots, K\}$ indexes stages, $j \in \{1, \ldots, m_k\}$ indexes the m_k relations in stage k, and z_t is the active stage at time t, with entry snapshot ξ_{z_t}. After verification, policy learning and data production use the accepted program $\Gamma^\star$.

A. Grounding passes and active operator vocabulary

SUN Program formation uses three structured transformations. First, the instruction is decomposed into `[Approach]`, `[Manipulate]`, and `[Release]` subtasks. Each subtask contains one controller-stage-sized change—one spatial change or one gripper-state change—together with simultaneous pose or grasp requirements. Second, an atomic-grounding pass resolves the current subtask into frame-, axis-, distance-, and feedback-specific language. Third, the constraint-writing pass selects and binds operators from $\mathcal{L}$ to produce one flat stage Γ_k for compilation. Appendix V provides the prompt contracts for these transformations and their diagnostic repair calls.

Grounding binds operators to names registered in $\mathcal{F}$. The harness uses tool $+z$ as the approach axis and tool $+y$ as the finger axis. A top-down approach makes tool $+z$ antiparallel to world $+z$; a level approach makes those axes orthogonal while pointing tool $+z$ at the target; and an axis-specific approach makes tool $+z$ antiparallel to the selected signed object axis. For an elongated grasp target, tool $+y$ is orthogonal to its long axis; for a box, it is orthogonal to the selected clamping axis. Rotational symmetry is otherwise left unconstrained.

The grounding prompt distinguishes clearance from contact. For moving- and reference-object extents l_c and l_r and a non-contact margin m, the requested center separation is

$$d_{\text{sep}} = \frac{l_c}{2} + \frac{l_r}{2} + m, \qquad m \geq 0.10 \text{ m}, \tag{5}$$

with $m \geq 0.15\,\text{m}$ when the instruction requests a "large" clearance, unless it provides a tighter value. Contact and grasp setup target the named frame origin or object center and specify the intended contact feedback. These separations define target geometry. The receiving application separately assigns the numerical tolerances used to test satisfaction.

Table VII lists the ten operations exposed by the active constraint-writing prompt. The implementation names `axis_align`, `directional_move`, and `freeze_orientation` correspond to `align_axis`, `direction_move`, and `fix_orientation` in Section III-A, respectively. The table retains the implementation names used by the prompt API. The `controlled_entity` is either a registered robot frame or a gripped object propagated through a fixed grasp transform. A reference basis supplies coordinates and axes, while a reference origin supplies the corresponding pose. Distances are in metres and angles are in radians. For all seven geometric operators, the receiving application supplies ϵ_p or ϵ_θ according to the operator and profile. These are the positional and angular instances of the relation tolerance $\epsilon_{k,j}$ and enter the compiled predicates through receiver-owned fields, which are omitted from the prompt's output schema. Per-term weights are assigned through a separate scalar-selection interface. Compilation initializes $w_{k,j} = 1$ unless that interface supplies a value; later calibration may change this weight without changing the accepted relation or its satisfaction boundary.

TABLE VII

PROMPT-VISIBLE SUN OPERATORS, THEIR GROUNDED ARGUMENTS, AND THEIR IMPLEMENTED SEMANTICS. THE RECEIVING APPLICATION ADDS NUMERICAL PREDICATE TOLERANCES BEFORE COMPILATION; THESE FIELDS ARE ABSENT FROM THE AGENT'S OUTPUT SCHEMA.

Operation	Principal parameters	Implemented meaning
`reach_point`	controlled entity/target, reference basis/origin, offset	Reach one frame-relative point; the intervening path is unconstrained.
`directional_move`	controlled entity/target, reference axis, $D > 0$	Translate by D along one signed axis while suppressing lateral drift.
`axis_align`	controlled and reference signed axes, relation	Enforce codirectional, antiparallel, bipolar, or orthogonal axes.
`point_toward`	controlled signed axis, target point	Aim an axis at a frame-relative point.
`directional_rotate`	reference signed axis, angle ϑ	Rotate from the stage-entry orientation by a specified amount.
`freeze_position`	controlled entity/target	Preserve the position captured on stage entry.
`freeze_orientation`	controlled entity/target	Preserve the orientation captured on stage entry.
`open_gripper`	none	Require the released/open mechanical state.
`close_gripper`	none	Require mechanical finger closure without claiming a grasp.
`object_grasped`	none	Require physical holding and retain it during later motion.

B. Receiver-owned tolerances and prompt filtering

A new or corrected agent response specifies operators, grounded targets, frames, distances, offsets, rotation amounts, and contact conditions. Before compilation, the receiving application assigns $\epsilon_{k,j}$ from the operator, stage semantics, effort conditions, and scene dimensions. This deterministic assignment establishes numerical satisfaction boundaries and records the selected profile and per-component decisions. The response schema excludes `tolerance_m` and `tolerance_rad`; any unsolicited values for these fields are ignored.

The implementation provides two deterministic initialization profiles. Fresh MPC formulation caps positional tolerances at 0.01 m, or 0.02 m for a nonzero-offset reach target, and sets the nominal angular field to 0.20 rad. Learning preparation instead derives positional fields in $[0.01, 0.05]$ m and angular fields in $[0.30, 0.50]$ rad according to geometry and interaction role. Saved MPC programs retain their recorded values on replay. Learning values are set before weight selection and remain fixed during later scalar tuning. These profiles use different numerical boundaries for MPC and learning preparation. In contrast, the reported experiments follow the fixed-boundary protocol in Algorithm 1, which retains the accepted program's satisfaction boundaries throughout policy learning.

Before each agent call, an outgoing-context filter removes receiver-owned setting fields and recognized tolerance annotations from prompt copies, including embedded JSON, correction diagnostics, and text history. The filter preserves geometric targets, numerical geometry, and unrecognized prose; the receiving application enforces tolerance assignment after the response. Stored programs and image attachments remain unchanged. Initial and BC-informed weight calls return `Diagnosis`, `Reasoning`, and `Weights`; an unsolicited `Parameters` field is ignored. Saved parameter artifacts retain the tolerance-assignment policy version and per-stage decisions for traceability.

C. Common frame and compiled-cost conventions

Let ${}^{W}p_A$ and ${}^{W}q_A$ be the world position and scalar-first unit quaternion of frame A, let $R(q) \in SO(3)$ be its rotation matrix, and let a superscript 0 denote a value stored in the current stage-entry snapshot ξ_k. The world superscript and time argument are omitted when unambiguous; thus p_c and q_c denote the controlled entity's world pose. A hat denotes unit normalization, $\otimes$ quaternion multiplication, and q^* quaternion conjugation. Rotation angles use ϑ, reserving θ for the Stage-BC parameters as in the main text. For local offset δ, reference origin O, and reference robot frame F, `reach_point` constructs

$$
{}^{W}p^{\star} = \begin{cases} {}^{W}p_O(t) + \delta, & \text{world basis,} \\ {}^{W}p_O(t) + R({}^{W}q_O(t))\delta, & \text{object basis,} \\ {}^{W}p_F^0 + R({}^{W}q_F^0)\delta, & \text{robot-frame basis.} \end{cases} \tag{6}
$$

In the world branch, δ is a world-coordinate offset from the selected origin; it is an absolute world point only when O is the reserved world frame.

When the controlled entity is an object held by end effector E, the entry snapshot defines the fixed grasp transform

$$
q_{O/E} = ({}^{W}q_E^0)^* \otimes {}^{W}q_O^0, \qquad r_{O/E} = R({}^{W}q_E^0)^\top ({}^{W}p_O^0 - {}^{W}p_E^0), \tag{7}
$$

and the optimized end-effector state implies

$$
{}^{W}q_O = {}^{W}q_E \otimes q_{O/E}, \qquad {}^{W}p_O = {}^{W}p_E + R({}^{W}q_E) r_{O/E}. \tag{8}
$$

The same position and orientation operations can therefore control the end effector or a rigidly attached object without adding object pose to the MPC decision state.

The implementation uses the quadratic MPC map and bounded learning map

$$
Q(v) = \frac{1}{2} v^\top v, \qquad B_a(r) = \frac{r^2}{r^2 + a^2}, \quad 0 \le B_a(r) < 1. \tag{9}
$$

Here $a > 0$ is the map's scale parameter. The rational map B_a bounds the learning cost, with lower c^{rl} indicating better satisfaction. The following equations instantiate the three compiled interfaces in Eq. (1) for each operator. Operator subscripts replace (k, j) for readability: for example, $\phi^{\mathrm{mpc}}_{\mathrm{reach}}$, P_{reach}, and $c^{\mathrm{rl}}_{\mathrm{reach}}$ denote $\phi^{\mathrm{mpc}}_{k,j}$, $P_{k,j}$, and $c^{\mathrm{rl}}_{k,j}$ for a point-reaching relation. Predicates use the same indicator notation $\mathbf{1}[\cdot]$ as the main text.

1) Translation operators: For `reach_point`, let p_c be the controlled robot-frame position or the composed object position from Eq. (8). Then

$$
\begin{aligned} e_{\mathrm{reach}} &= p_c - {}^{W}p^{\star}, & d_{\mathrm{reach}} &= \|e_{\mathrm{reach}}\|_2, \\ \phi^{\mathrm{mpc}}_{\mathrm{reach}} &= Q(e_{\mathrm{reach}}), & P_{\mathrm{reach}} &= \mathbf{1}[d_{\mathrm{reach}} \le \epsilon_p], \\ c^{\mathrm{rl}}_{\mathrm{reach}} &= B_{0.5}(d_{\mathrm{reach}}). \end{aligned} \tag{10}
$$

The tolerance ϵ_p sets the satisfaction boundary; the MPC objective and RL learning cost retain the forms above.

For `directional_move`, resolve the signed basis axis as world unit vector $\widehat{a}$, let $D > 0$ be the requested travel, and measure from the controlled entity's entry position $p_c^0 \in \xi_k$:

$$\begin{aligned} \Delta p &= p_c - p_c^0, & d_\parallel &= \widehat{a}^\top \Delta p, & e_\parallel &= d_\parallel - D, \\ d_\perp &= \Delta p - \widehat{a} d_\parallel, & m_{\text{move}} &= \sqrt{e_\parallel^2 + \tfrac{1}{2}\|d_\perp\|_2^2}. \end{aligned} \tag{11}$$

Its compiled views are

$$\begin{aligned} &\phi_{\text{move}}^{\text{mpc}} = \tfrac{1}{2}(e_\parallel^2 + \|d_\perp\|_2^2), \\ &P_{\text{move}} = \mathbf{1}[m_{\text{move}} \le \epsilon_p], \qquad c_{\text{move}}^{\text{rl}} = B_{D/2}(m_{\text{move}}). \end{aligned} \tag{12}$$

All three views use the same entry position, travel direction, and requested distance. Their numerical penalties differ: the predicate and RL learning cost weight squared lateral deviation by one half within m_{move}^2, while the MPC objective uses its full value.

2) Orientation operators: For selected signed canonical axes $\widehat{e}_i$ and $\widehat{e}_\ell$, define

$$\widehat{a}_c = \widehat{R(^W q_c)\widehat{e}_i}, \qquad \widehat{a}_r = \widehat{R(^W q_r)\widehat{e}_\ell}, \qquad s = \widehat{a}_c^\top \widehat{a}_r, \tag{13}$$

where the reference axis may instead be a fixed world direction. The `axis_align` relation chooses

relation	$\eta(s)$	$\bar{\epsilon}(\epsilon_\theta)$
codirectional	$1 - s$	$1 - \cos\epsilon_\theta$
antiparallel	$1 + s$	$1 - \cos\epsilon_\theta$
bipolar	$1 - \lvert s\rvert$	$1 - \cos\epsilon_\theta$
orthogonal	$\sqrt{s^2 + 10^{-18}}$	$\sin\epsilon_\theta$

(14)

and compiles it as

$$\begin{aligned} \phi_{\text{align}}^{\text{mpc}} &= Q(\eta), & P_{\text{align}} &= \mathbf{1}[\eta \le \bar{\epsilon}], \\ c_{\text{align}}^{\text{rl}} &= \begin{cases} B_{0.25}(\eta/2), & \text{co-, anti-, or bipolar,} \\ B_{0.25}(|\eta|), & \text{orthogonal.} \end{cases} \end{aligned} \tag{15}$$

For `point_toward`, the selected axis aims from the end effector toward the target point:

$$\widehat{d} = \widehat{{}^W p^\star - {}^W p_E}, \qquad \eta_{\text{point}} = 1 - \widehat{a}_c^\top \widehat{d}, \tag{16}$$

$$\begin{aligned} &\phi_{\text{point}}^{\text{mpc}} = Q(\eta_{\text{point}}), \\ &P_{\text{point}} = \mathbf{1}[\eta_{\text{point}} \le 1 - \cos\epsilon_\theta], \qquad c_{\text{point}}^{\text{rl}} = B_{0.25}(\eta_{\text{point}}/2). \end{aligned} \tag{17}$$

The MPC interface binds the controlled axis to a robot frame and constructs the target as ${}^W p_O(t) + \delta$, treating the offset as world coordinates regardless of the supplied basis. The batched predicate additionally supports a gripped object's orientation for $\widehat{a}_c$. In both cases, $\widehat{d}$ originates at the configured end effector.

For `directional_rotate`, let $\vartheta > 0$ be the requested angle, $\widehat{a}$ the signed axis in the selected basis, and q_F^0 the world orientation of its reference frame stored in ξ_k. Then

$$q_\Delta = \left(\cos\frac{\vartheta}{2}, \widehat{a}\sin\frac{\vartheta}{2}\right), \qquad {}^W q_\Delta = \begin{cases} q_F^0 \otimes q_\Delta \otimes (q_F^0)^*, & \text{object or robot-frame basis,} \\ q_\Delta, & \text{world basis,} \end{cases} \tag{18}$$

and, for controlled entry orientation q_c^0,

$$\begin{aligned} q^\star &= {}^W q_\Delta \otimes q_c^0, & e_q &= q_c - q^\star, \\ \phi_{\text{rotate}}^{\text{mpc}} &= Q(e_q), & P_{\text{rotate}} &= \mathbf{1}[\|e_q\|_2 < \epsilon_\theta], \\ c_{\text{rotate}}^{\text{rl}} &= B_{0.25}(\|e_q\|_2). \end{aligned} \tag{19}$$

This predicate applies its nominal angular tolerance to quaternion-coordinate distance, rather than to a geodesic angle; the distance is sensitive to the equivalent quaternion signs q and $-q$. The formation prompt places `axis_align` relations involving the rotating frame in an earlier approach stage and the finite rotation in the subsequent manipulation stage. These relations therefore occupy separate stages.

3) Entry invariants and gripper operators: The freeze operators preserve poses captured in the stage-entry snapshot. Position freezing uses

$$\begin{aligned} e_f^p &= p_c(t) - p_c^0, \qquad \phi_{f,p}^{\mathrm{mpc}} = Q(e_f^p), \\ P_{f,p} &= \mathbf{1}[\|e_f^p\|_2 \le \epsilon_p], \quad c_{f,p}^{\mathrm{rl}} = B_{0.25}(\|e_f^p\|_2). \end{aligned} \tag{20}$$

For orientation, the predicate and learning path use the sign-invariant geodesic distance

$$\begin{aligned} \vartheta_f &= 2\arccos\big(\mathrm{clip}(|\widehat{q}_c(t)^\top \widehat{q}_c^0|, 0, 1)\big), \\ P_{f,q} &= \mathbf{1}[\vartheta_f \le \epsilon_\theta], \qquad c_{f,q}^{\mathrm{rl}} = B_{0.25}(\vartheta_f). \end{aligned} \tag{21}$$

The MPC objective uses

$$\phi_{f,q}^{\mathrm{mpc}} = Q(q_E - q_E^0) \tag{22}$$

and evaluates orientation changes at the configured end-effector state frame, independently of the operation's controlled-entity and target arguments.

For normalized optimizer gripper state $g = (g_L, g_R)$,

$$g_{\mathrm{closed}} = (0,0), \quad g_{\mathrm{open}} = (1,1), \quad \phi_\sigma^{\mathrm{mpc}} = Q(g - g_\sigma), \quad c_\sigma^{\mathrm{rl}} = B_1(\|g - g_\sigma\|_2), \tag{23}$$

for gripper mode $\sigma \in \{\mathrm{closed}, \mathrm{open}\}$. Measured scalar openness o_{grip} is normalized from the n_g finger-joint positions q_ℓ and their calibrated limits:

$$o_{\mathrm{grip}} = \frac{1}{n_g}\sum_{\ell=1}^{n_s} \frac{q_\ell - q_\ell^{\mathrm{closed}}}{q_\ell^{\mathrm{open}} - q_\ell^{\mathrm{closed}}}, \quad P_{\mathrm{close}} = \mathbf{1}[o_{\mathrm{grip}} \le 0.1], \quad P_{\mathrm{open}} = \mathbf{1}[o_{\mathrm{grip}} \ge 0.9]. \tag{24}$$

Physical holding is checked using finger contact and openness in addition to the close-gripper command. Let F_L, F_R be the filtered finger-contact magnitudes, $F_{\max}$ the configured maximum finger-force scale, and $o_{\max}$ the maximum grasp openness. Define

$$\begin{aligned} o_c &= \mathrm{clip}(o_{\mathrm{grip}}, 0.03, o_{\max}), & \widetilde{o}_{\mathrm{grip}} &= \frac{o_c - 0.03}{o_{\max} - 0.03}, \\ F_{\mathrm{req}}(o_{\mathrm{grip}}) &= F_{\max}(0.1 + 0.6\widetilde{o}_{\mathrm{grip}}), & P_{\mathrm{width}} &= \mathbf{1}[0.1 < o_{\mathrm{grip}} \le o_{\max}], \\ P_{\mathrm{both}} &= \mathbf{1}[F_L \ge F_{\mathrm{req}}] \wedge \mathbf{1}[F_R \ge F_{\mathrm{req}}], \\ P_{\mathrm{one}} &= (\mathbf{1}[F_L \ge 10] \oplus \mathbf{1}[F_R \ge 10]) \wedge \mathbf{1}[o_{\mathrm{grip}} < 0.3] \wedge P_{\mathrm{width}}. \end{aligned} \tag{25}$$

The robot-specific predicate is

$$P_{\mathrm{grasp}} = \begin{cases} (P_{\mathrm{both}} \wedge P_{\mathrm{width}}) \vee P_{\mathrm{one}}, & \text{Boston Franka or Kinova Robotiq,} \\ \mathbf{1}[F_L \ge 5] \wedge \mathbf{1}[F_R \ge 5] \wedge P_{\mathrm{width}}, & \text{Slichter Franka Panda.} \end{cases} \tag{26}$$

The Boston Franka and Kinova Robotiq predicates include the one-finger exception P_{one}, whereas the Slichter Franka Panda predicate requires contact at both fingers. The MPC objective and RL learning cost for `object_grasped` reuse the close-gripper costs in Eq. (23); its satisfaction predicate additionally checks physical holding. By default, a stage containing this operation must remain satisfied for five consecutive frames before advancing. Later pulling, carrying, and reorientation stages retain the operation to monitor continued holding.

D. Condition labels and valid term composition

Condition labels attach satisfaction or feedback semantics to an operation. `geometry` selects its geometric predicate; `gripper_opened` and `gripper_closed` describe finger state; and `manipulator_high_effort` and `manipulator_low_effort` describe arm/TCP interaction rather than finger squeeze. For exponentially filtered arm effort and a configured high-effort threshold E_{high},

$$\bar{\tau}_t = 0.2\tau_t + 0.8\bar{\tau}_{t-1}, \qquad E_t = \sum_{\ell=1}^{7} |\bar{\tau}_{t,\ell}|, \qquad P_{\mathrm{low}} = \mathbf{1}[E_t \le 10], \tag{27}$$

and sustained high effort is

$$h_t^E = \begin{cases} \min(h_{t-1}^E + 1, 5), & E_t > E_{\mathrm{high}}, \\ 0, & E_t \le E_{\mathrm{high}}, \end{cases} \qquad P_{\mathrm{high}} = \mathbf{1}[h_t^E \ge 5]. \tag{28}$$

Effort labels contribute to satisfaction checks through local stopping conditions within a required term, rather than through standalone MPC objectives or RL learning costs. For a relation that permits high effort as a local stopping alternative, its compiled predicate and the stage-completion condition are

$$P_{k,j}(t) = P_{k,j}^{\text{geometry}}(t) \vee P_{\text{high}}(t), \qquad D_k(t) = \bigwedge_{j=1}^{m_k} P_{k,j}(t). \tag{29}$$

Here D_k is the conjunction of the relation predicates. As in the main text, the monitor evaluates $D_{z_t}(t+1)$ after applying u_t and before updating the active stage. A stage contains at most one of `reach_point`, `directional_move`, and `freeze_position`; a freeze operation cannot substitute for a new pose goal; and `close_gripper` and `object_grasped` cannot co-occur. `directional_move` is limited to manipulation or release stages and requires an explicit direction and positive distance. When a required clause falls outside $\mathcal{L}$, the agent records that clause as unsupported and omits it from the executable terms. The runtime also supports legacy `reach_distance` and `free_move` operators for replaying older saved programs; the active prompt uses the vocabulary in Table VII.

At run time, diagnostics retain the source stage, bound frames, axis, tolerance, measured residual, and satisfaction margin. After executing one action, the monitor observes the resulting state, evaluates the current-stage predicates and independent external outcome, computes learning and diagnostic signals, and then updates the active stage. Each new stage captures its reference poses after the guarded transition, preserving the evaluation order used during verification, learning, and data production.

E. Policy and action implementation

Both learned state-policy components use feed-forward multilayer perceptrons. The Stage-BC prior $\pi_\theta(o_t, z_t)$ is a deterministic MLP, and the residual policy $\pi_\psi(o_t, z_t)$ is trained with PPO and generalized advantage estimation using a feed-forward actor–critic. Here o_t is the privileged observation, distinct from scalar gripper openness o_{grip}. The absolute action u_t in Eq. (2) has the implementation representation

$$u_t = \begin{bmatrix} p_t^\top & (R_t^{6\text{D}})^\top & g_t^\top \end{bmatrix}^\top \in \mathbb{R}^{11}, \tag{30}$$

where $p_t \in \mathbb{R}^3$ is the Cartesian position target, $R_t^{6\text{D}} \in \mathbb{R}^6$ is the continuous six-dimensional rotation representation of the orientation target, and $g_t \in \mathbb{R}^2$ contains the two finger targets used to encode the gripper command.

Stage-BC predicts chunks of ten actions and is trained with normalized mean-squared error:

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N} \sum_{n=1}^{N} \left\| D_u^{-1} (\widehat{U}_n(\theta) - U_n) \right\|_2^2, \tag{31}$$

where N is the number of training chunks, U_n stacks the target actions, and $\widehat{U}_n(\theta)$ is the chunk predicted from the privileged observation and active stage. The diagonal matrix D_u contains the action-normalization scales repeated across the chunk. Chunks never cross a stage boundary; when fewer than ten actions remain, the last action in that stage is repeated as padding. The first predicted action is $\pi_\theta(o_t, z_t)$; its bounded composition with the residual gives $u_t = \pi_{\text{pri}}(o_t, z_t)$ in Eq. (2), after which the SUN monitor is reevaluated.

Appendix III reports the architectures and temporal targets evaluated in the task-specific BC sweep. Reproducing a final Kuafu controller also requires its selected architecture, optimizer and epoch schedule, normalization constants, PPO/GAE hyperparameters, residual bounds, reward weights, MPC horizon and control rate, solver tolerances, and domain-randomization ranges. Complete per-task values for those final controller settings are not reported here; the sweep configurations describe the supplemental BC comparisons.

APPENDIX III
BC-PRIOR CAPACITY AND TEMPORAL-TARGET SWEEP

Table VIII reports the supplemental BC-prior sweep on T6 (three-cube stacking) and T8 (cup hanging). It varies the feed-forward policy architecture, supervised action-chunk length C, and spacing s_a between actions in the training target. This evaluates BC alone, not the complete residual-RL controller. Model names retain the task-specific sweep identifiers: their hidden-layer widths are stated explicitly, and the suffix `3` denotes a three-hidden-layer variant, not an action stride.

TABLE VIII

BC POLICY-CAPACITY, ACTION-CHUNK, AND STRIDE SWEEP. HIDDEN-LAYER WIDTHS ARE LISTED IN INPUT-TO-OUTPUT ORDER. ROLLOUT SUCCESS RATES (%) ARE GROUPED BY SUPERVISED ACTION-CHUNK LENGTH C AND SAMPLING STRIDE s_a. DASHES RETAIN UNREPORTED CONFIGURATIONS, NOT ZERO SUCCESS; BOLD MARKS THE BEST REPORTED ENTRY FOR EACH TASK.

Model	Hidden-layer widths	$C = 10$		$C = 15$		$C = 30$	
		$s_a = 1$	$s_a = 3$	$s_a = 1$	$s_a = 3$	$s_a = 1$	$s_a = 3$
T6: Stack 3 Cubes							
`ff`	2048, 1024, 512, 1024, 2048	0.6	2.7	1.5	3.4	3.5	3.0
`fff`	4096, 2048, 1024, 2048, 4096	7.3	7.4	10.1	14.2	10.2	11.3
`fff3`	2048, 1024, 2048	2.0	5.0	3.1	4.9	4.1	0.5
`ffff`	8192, 4096, 2048, 4096, 8192	16.1	23.6	**30.0**	27.8	22.5	18.4
`ffff3`	4096, 2048, 4096	4.7	6.8	6.3	6.4	6.9	3.2
`fffff`	16384, 8192, 4096, 8192, 16384	7.4	17.3	16.9	25.1	26.5	0.1
`fffff3`	8192, 4096, 8192	6.1	10.3	3.0	11.6	5.8	8.8
`ffffff`	32768, 16384, 8192, 16384, 32768	0.1	0.3	0.1	6.7	11.3	4.5
`ffffff3`	16384, 8192, 16384	3.8	1.1	2.2	6.3	3.2	3.5
T8: Hang Cup							
`ff`	2048, 1024, 512, 1024, 2048	3.2	2.0	4.9	1.3	1.6	0.0
`fff`	4096, 2048, 1024, 2048, 4096	17.7	3.4	9.1	3.3	5.4	2.1
`fff3`	2048, 1024, 2048	12.5	8.6	13.5	6.2	8.8	4.3
`ffff`	8192, 4096, 2048, 4096, 8192	**22.5**	–	17.9	–	–	–
`ffff3`	4096, 2048, 4096	19.2	–	17.8	–	–	–
`fffff`	16384, 8192, 4096, 8192, 16384	20.2	–	12.7	–	3.7	–
`ffffff`	32768, 16384, 8192, 16384, 32768	14.8	–	16.6	–	7.6	–

The sweep reveals task-dependent effects of capacity and temporal targets. Among the reported configurations, the intermediate-sized `ffff` architecture achieves the highest success on both tasks: 30.0% on T6 with $C = 15$, $s_a = 1$, and 22.5% on T8 with $C = 10$, $s_a = 1$. Increasing width beyond this architecture reduces the best observed success on each task, while the different optimal chunk lengths indicate that temporal targets should be selected for the task. The stride comparison further distinguishes the two tasks: increasing s_a from 1 to 3 improves 19 of the 27 matched T6 configurations, but lowers success in all nine reported matched T8 configurations. The highest-success configuration for each task nevertheless uses $s_a = 1$.

These results support task-specific selection of the BC prior's architecture and temporal targets. Even the strongest tested BC configurations retain substantial execution failures on these two tasks. The sweep therefore characterizes the starting behavior available to subsequent residual refinement, complementing the complete-system ablations in Section IV-B.

APPENDIX IV
TASKWISE DATA-GENERATION THROUGHPUT AND AMORTIZATION

Figures 3 and 4 expand the throughput and break-even summaries in Table IV, panels A and B, with taskwise results from the same evaluation. They distinguish steady-state production from the upfront time and monetary costs of preparing the final controller. Plot annotations are rounded; the table retains the reported numerical precision.

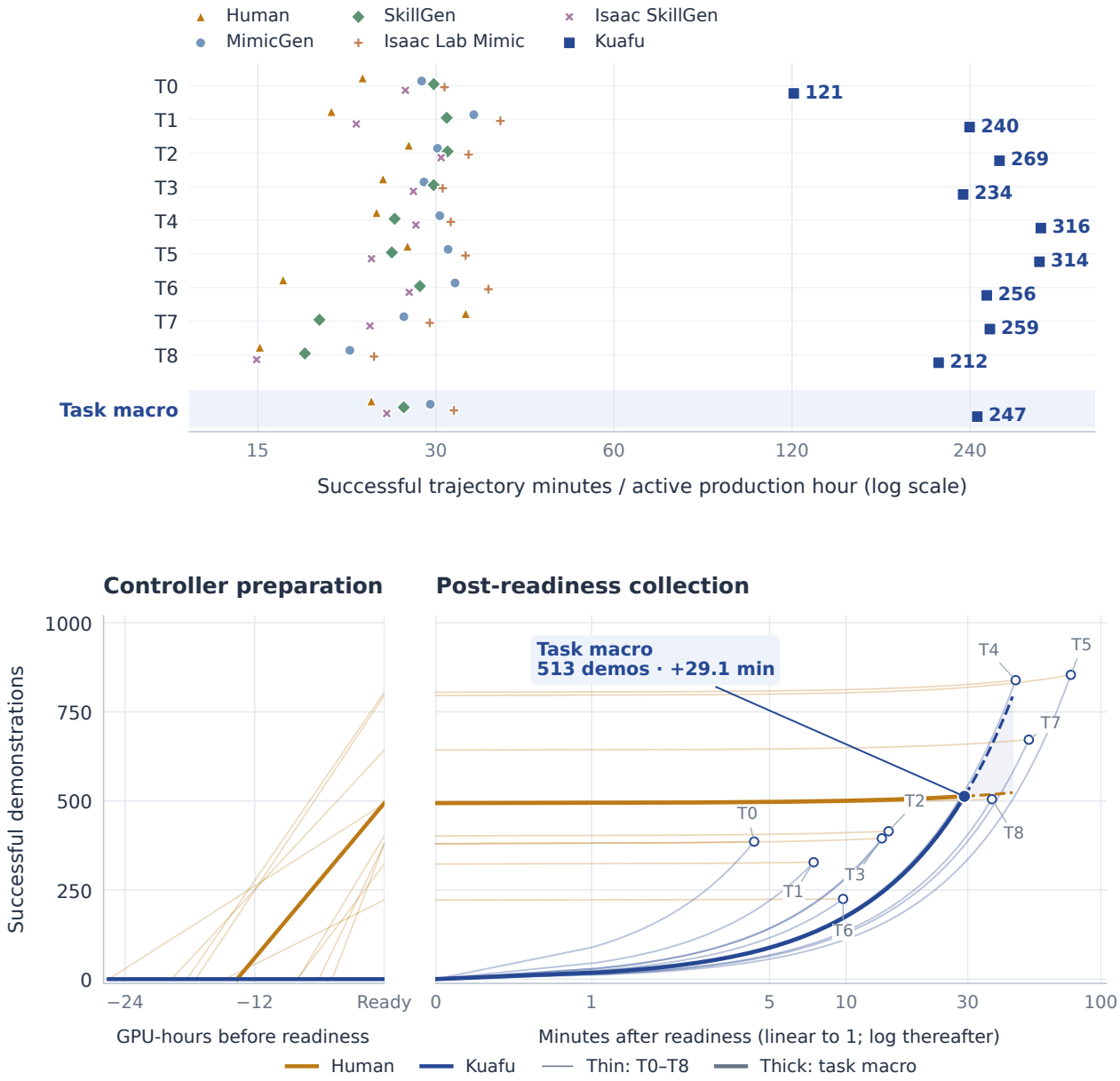


Fig. 3. Taskwise data-generation throughput and startup break-even. **Top:** successful trajectory minutes per active production hour for tasks T0–T8 and their unweighted task-macro average; the horizontal axis is logarithmic. Production hours refer to active human collection for the human reference and GPU production for the simulated generators. **Bottom:** cumulative successful demonstrations reconstructed from the reported readiness and time-break-even values. Faint curves show individual tasks; thick curves show the reported task-macro summary, and dashed extensions project continued collection at the same rates. The horizontal axis uses GPU-hours before readiness and minutes afterward; the post-readiness axis is linear up to one minute and logarithmic thereafter. The macro break-even is 513 demonstrations after 29.1 minutes of post-readiness collection.

Kuafu's throughput advantage extends across all nine tasks, consistent with the $10.57\times$ task-macro gain over human teleoperation in Table IV. Including controller preparation, time break-even occurs at 225–854 successful demonstrations across tasks, below 1,000 on every task. The macro crossover is 513 demonstrations after 29.1 minutes of post-readiness collection. Thus, higher production throughput amortizes the one-time controller-preparation effort as the dataset grows.

A. *Cost parity*

The monetary comparison uses the same pricing assumptions as the main text: $7.25 per hour of human teleoperation and $0.51 per GPU-hour for both controller preparation and data production on an A6000. For an equal number of successful demonstrations, human cost scales with active collection time, whereas Kuafu's cost includes controller preparation plus GPU collection time. Per-demonstration rates are computed separately for each task from its reported collection counts and durations. Figure 4 shows the resulting cost curves and the first whole-demonstration count at which Kuafu is no more expensive than human teleoperation.



Fig. 4. Taskwise cost parity under the main text's pricing assumptions. **(a)** Modeled costs at matched successful-demonstration counts. Thin curves and open circles show individual tasks up to their continuous cost crossings; thick curves show costs averaged across tasks, including Kuafu's preparation cost. **(b)** The first whole-demonstration count reaching cost parity for each task and the mean of these thresholds. Averaging taskwise thresholds yields 35.22 demonstrations, whereas intersecting the mean-cost curves in (a) yields approximately 32.38; these are distinct summaries. The footer reports taskwise averages, with collection time measured after controller readiness.

Cost parity occurs after 16–57 successful demonstrations across the nine tasks, averaging 35.22 demonstrations and 1.99 minutes of post-readiness collection; the latter ranges from 0.30 to 5.09 minutes. The mean controller-preparation cost is $6.95 and is included in these thresholds. Under the stated hourly rates, monetary parity therefore occurs at a smaller dataset size than time parity: the lower GPU collection cost recovers the preparation expenditure before Kuafu overtakes the human collector in cumulative demonstration count.

APPENDIX V
SUN PROMPT SPECIFICATIONS

This appendix presents the prompt templates for SUN Program formation and policy calibration in the Kuafu harness. The prompts and accompanying source code will be made publicly available on GitHub. Braced fields are replaced at run time by scene data, task state, diagnostics, or learning statistics. The templates are reproduced verbatim, without run-specific conversations, model responses, or private evaluator definitions. They describe the current implementation; differences from the reported experimental protocol are identified in Appendix II-B.

Table IX maps each agent call to the evidence it receives. "System" denotes the persistent behavioral contract and "user" denotes the run-time template. Formation calls emit a JSON object. The current implementation uses the agent for these calls, with sampling controls left at the provider defaults.

TABLE IX
PROMPT INVENTORY FOR SUN FORMATION AND LEARNING.

Harness stage	Prompt pair	Run-time evidence inserted
Task decomposition	Initial system and user templates	Instruction, completed subtasks, pending subtasks
Decomposition repair	Correction system and user templates	Previous decomposition and execution error
Atomic grounding	Atomic-stage system and user templates	Current subtask, robot state/control descriptions, operator API
Typed relation writing	Constraint system and user templates	Atomic action, registered frames, completed/pending context
Relation repair	Constraint-correction system and user templates	Previous grounding, type or execution error, predicate diagnostics
Initial weight selection	Initial-sweep system and user templates	Stage semantics, neighboring stages, geometry-only operator arguments, weights
BC-prior weight adjustment	BC-diagnostic system and stage user templates	BC-prior outcomes, satisfaction times, timeout/stuck component rates, reward values
Residual-authority calibration	Residual system and user templates	Action dimensions, outcome counts, stage passage, training traces, collapse plot

A. Program-formation prompts

The five prompt pairs below expand the grounding and verification phase of Algorithm 1, covering the three formation transformations and their two diagnostic return paths. Their wording and placeholder names match those used by the implementation.

1) Initial task decomposition: Listing V.1: System context for initial task decomposition

```
You write pre-task manipulation plans as phase-labeled planner subtasks. Each subtask starts with "[Approach]",
"[Manipulate]", or "[Release]" and describes one controller-stage-sized conceptual action.

Inputs include the top-level task, observations, visible object names, robot frames/joints, previously seen objects, and an
Important rule limiting references to listed objects/frames/locations.

Priority rules:
1. Output exactly one JSON object with keys "Reasoning" and "Subtasks". No extra text, code fences, or backticks.
2. "Subtasks" is an ordered string array. Each subtask must be executable as one later controller stage: one spatial change OR
one gripper-state change, plus any required fixed-position/fixed-orientation condition. Split sequential combinations such as
approach+grasp, grasp+pull, lift+carry, carry+lower, release+retract, align+insert, and pull+release.
3. Use only listed objects, frames, and locations. Use exact names; listed `_frame` names are allowed task targets. Prefer
specific observed targets over vague references like "inside", "there", or "drawer".
4. Do not mention coordinates, axes, coordinate-frame mechanics, numeric distances/speeds/angles, or low-level geometric terms
such as codirectional, antiparallel, bipolar, orthogonal, collinear, or coaxial.
5. For grasp-then-move/pull interactions, follow the GRASP pattern. For contact-without-grasp interactions, follow the CONTACT
pattern. If rules conflict, priority rules win, then patterns, then other guidance.

Scope:
- Plan conceptually: describe WHAT the robot does, not controller details or trajectories.
- If manipulating through the scene, include a clearance-creating [Manipulate] subtask before carrying/repositioning when
surfaces, obstacles, or receptacles matter.
- One subtask must not hide a sequence.

Approach label selection:
Every [Approach] subtask contains exactly one of: "top-down approach", "leveled approach", "direct approach toward the
object", or "axis approach". Choose by the immediate interaction target/feature, not only the object category. Use the first
matching rule:
1. "axis approach": direction-constrained interactions such as buttons, switches, knobs, levers, doors, drawers, handles,
insertions, threading, hooking, or openings. Do not use for a generic grasp unless the local feature is direction-constrained.
2. "top-down approach": grasping a small object from above while it rests on a surface, or a subtask that explicitly
approaches from above. Do not use for side grasps of upright objects/features.
3. "leveled approach": side interaction with a tall, upright, or standing object/body.
4. "direct approach toward the object": fallback when no stronger cue exists; briefly state finger orientation relative to the
target without coordinates/frames.

GRASP pattern for picking, moving, carrying, placing, or pulling by grasping <obj>:
1. "[Approach] Use an appropriate <approach label> to move the open gripper <relation> <obj> without touching it." Use "above"
for top-down and "near" otherwise.
2. "[Approach] Move the open gripper into contact with <obj> while keeping the gripper orientation fixed." This step must not
include standoff.
3. "[Manipulate] Close the gripper to grasp <obj> while keeping both the gripper position and orientation fixed."
4. Move the grasped object with the gripper closed. Split multi-stage motion into clearance, carry/align, and optional lower/
place subtasks. Do not combine lift, carry, align, and lower. If engagement is needed, split into clearance when transporting,
alignment-only, then engagement-only; for insertion/threading/sliding through an opening, the alignment-only subtask must
explicitly say "<obj> aligned with the opening of <target> for insertion" and must not use vague words like suitable/
appropriate.
5. "[Release] Open the gripper to release <obj> at the desired state or location while keeping both the gripper position and
orientation fixed."
6. "[Release] Retract the gripper away from <obj> to a non-contact configuration with the gripper open, preserving continuity
with the immediately preceding interaction."
Keep milestone order. Omit release/retract only if the task does not require releasing the object.

CONTACT pattern for pressing, pushing, tapping, depressing, or brief contact without grasping <obj>:
1. "[Approach] Use an appropriate <approach label> to move the end-effector near <obj> without touching it, with the gripper
closed." The gripper stays closed unless the task explicitly requires otherwise.
2. "[Approach] Move the end-effector into contact with <obj> while keeping the gripper orientation fixed."
3. "[Manipulate] Perform the required contact motion on <obj> to achieve the desired state, stating whether orientation,
position, or both stay fixed." Do not force push/press verbs unless required.
4. "[Release] Retract the end-effector away from <obj> to a non-contact configuration, preserving continuity with the
immediately preceding interaction."
Keep milestone order and required fixed-orientation phrase.

Reasoning: 2-6 sentences stating the strategy, interactions, phase decomposition, approach-label justifications, required
fixed orientation/pose points, and any clearance expansion.

Output shape:
{
  "Reasoning": "...",
  "Subtasks": ["[Approach] ...", "[Manipulate] ..."]
}
```

Listing V.2: User template for initial task decomposition

```
Top-level task: {task_description}

Inputs:
- Optional scene image including the robot
- Robot frames / joints: {frames}
- Observed objects: {history}
- Important: reference only objects, frames, and locations listed in the inputs.

Decompose the top-level task into phase-labeled subtasks.

Requirements:
- Each subtask starts with exactly one phase tag: "[Approach]", "[Manipulate]", or "[Release]".
- Each subtask describes one controller-stage-feasible conceptual action: one spatial change OR one gripper-state change, plus
any required fixed-position/fixed-orientation condition.
- Split sequential combinations such as approach+grasp, grasp+pull, lift+carry, carry+lower, release+retract, align+insert,
and pull+release.
- Follow the system instructions for approach labels and GRASP / CONTACT patterns.
- Prefer exact observed target names, including listed `_frame` names, over vague references like "inside", "there", or
"drawer".

Return exactly one JSON object:
- "Reasoning": 2-6 sentences describing the strategy and decomposition.
- "Subtasks": ordered array of phase-labeled subtasks.

Reply with JSON only: double-quoted keys/strings, no extra text, no code fences, no backticks.
```

2) Task-decomposition correction: Listing V.3: System context for decomposition correction

```
You correct pre-task manipulation plans as phase-labeled planner subtasks. Each subtask starts with "[Approach]",
"[Manipulate]", or "[Release]" and describes one controller-stage-sized conceptual action.

Inputs include the top-level task, observations, visible object names, robot frames/joints, previously seen objects, execution
log/completed subtasks, an untrusted previous plan, pending/completed subtasks from that plan, an error/issue, and an
Important rule limiting references to listed objects/frames/locations.

Priority rules:
1. Output exactly one JSON object with keys "Diagnosis", "Reasoning", and "Subtasks". No extra text, code fences, or
backticks.
2. Treat the execution log as authoritative. Do not repeat completed actions; continue from the current state.
3. "Subtasks" is an ordered string array. Each subtask must be executable as one later controller stage: one spatial change OR
one gripper-state change, plus any required fixed-position/fixed-orientation condition. Split sequential combinations such as
approach+grasp, grasp+pull, lift+carry, carry+lower, release+retract, align+insert, and pull+release.
4. Use only listed objects, frames, and locations. Use exact names; listed `_frame` names are allowed task targets. Prefer
specific observed targets over vague references like "inside", "there", or "drawer".
5. Do not mention coordinates, axes, coordinate-frame mechanics, numeric distances/speeds/angles/counts, or low-level
geometric terms such as codirectional, antiparallel, bipolar, orthogonal, collinear, or coaxial.
6. For grasp-then-move/pull interactions, follow the GRASP pattern. For contact-without-grasp interactions, follow the CONTACT
pattern. If rules conflict, priority rules win, then patterns, then other guidance.

Correction scope:
- Keep compliant subtasks verbatim when possible.
- Make the smallest set of changes needed to fix invalid references, missing pattern steps/phrases, missing clearance before
manipulation-through-scene, wrong labels/order, execution-log conflicts, and the stated error.
- Plan conceptually: describe WHAT the robot does, not controller details or trajectories.

Approach label selection:
Every [Approach] subtask contains exactly one of: "top-down approach", "leveled approach", "direct approach toward the
object", or "axis approach". Choose by the immediate interaction target/feature. Use the first matching rule:
1. "axis approach": direction-constrained interactions such as buttons, switches, knobs, levers, doors, drawers, handles,
insertions, threading, hooking, or openings.
2. "top-down approach": grasping a small object resting on a surface or a subtask that explicitly approaches from above; not
for side grasps of upright objects/features.
3. "leveled approach": side interaction with a tall, upright, or standing object/body.
4. "direct approach toward the object": fallback; briefly state finger orientation relative to the target without coordinates/
frames.

GRASP pattern for picking, moving, carrying, placing, or pulling by grasping <obj>:
1. "[Approach] Use an appropriate <approach label> to move the open gripper <relation> <obj> without touching it." Use "above"
for top-down and "near" otherwise.
2. "[Approach] Move the open gripper into contact with <obj> while keeping the gripper orientation fixed." This step must not
include standoff.
3. "[Manipulate] Close the gripper to grasp <obj> while keeping both the gripper position and orientation fixed."
4. Move the grasped object with the gripper closed. Split multi-stage motion into clearance, carry/align, and optional lower/
place subtasks. Do not combine lift, carry, align, and lower. If engagement is needed, split into alignment-only then
engagement-only; do not combine alignment and insert/hook/thread/slide engagement.
5. "[Release] Open the gripper to release <obj> at the desired state or location while keeping both the gripper position and
orientation fixed."
6. "[Release] Retract the gripper away from <obj> to a non-contact configuration with the gripper open, preserving continuity
```

```
with the immediately preceding interaction."
Keep milestone order. Omit release/retract only if not needed by the remaining task.

CONTACT pattern for brief contact without grasping <obj>:
1. "[Approach] Use an appropriate <approach label> to move the end-effector near <obj> without touching it in a chosen gripper
state." Explicitly choose open/closed as contact posture; this is not a grasp unless the robot must hold the object.
2. "[Approach] Move the end-effector into contact with <obj> while keeping the gripper orientation fixed."
3. "[Manipulate] Perform the contact action on <obj> to achieve the desired state, stating whether orientation, position, or
both stay fixed."
4. "[Release] Retract the end-effector away from <obj> to a non-contact configuration, preserving continuity with the
immediately preceding interaction."
Keep milestone order and required fixed-orientation phrase.

Diagnosis: exactly "Evidence: ...; Cause: ...; Fix: ..." and cite concrete violations/mismatches.
Reasoning: 2-6 sentences stating the updated strategy, interactions, phase decomposition, approach-label justifications, fixed
orientation/pose points, and any clearance split.

Output shape:
{
  "Diagnosis": "Evidence: ...; Cause: ...; Fix: ...",
  "Reasoning": "...",
  "Subtasks": ["[Approach] ...", "[Manipulate] ..."]
}
```

Listing V.4: User template for decomposition correction

```
Top-level task: {task_description}

Observations and execution state:
* Optional scene image(s): {images}
* Robot frames / joints: {frames}
* Observed objects: {history}
* Execution log / completed subtasks: {completed}

Previous pre-task plan, untrusted:
* Reasoning: {planner_reasoning}
* Subtasks: {planner_subtasks}
* Pending subtasks: {pending}
* Completed subtasks from original plan: {completed_plan}

Error / Issue:
* Description: {error}

Using the system instructions, return a corrected phase-labeled plan.

Requirements:
- Diagnose the issue.
- Do not repeat completed actions.
- Keep valid previous subtasks when possible.
- Each corrected subtask must be one controller-stage-feasible conceptual action: one spatial change OR one gripper-state
change, plus any required fixed-position/fixed-orientation condition.
- Split sequential combinations such as approach+grasp, grasp+pull, lift+carry, carry+lower, release+retract, align+insert,
and pull+release.

Return exactly one JSON object:
- "Diagnosis": "Evidence: ...; Cause: ...; Fix: ..."
- "Reasoning": explanation of the correction.
- "Subtasks": ordered array of corrected phase-labeled subtasks.

Reply with JSON only: double-quoted keys/strings, no extra text, no code fences, no backticks.
```

3) Atomic-stage grounding: Listing V.5: System context for atomic-stage grounding

```
You ground ONE current phase-labeled planner subtask into exactly ONE natural-language grounded stage requirement for a
downstream constraint writer. You never write code or TermAPI JSON.

Inputs include the top-level task, current subtask, completed/pending subtasks for continuity, observations, robot frames,
observed objects/poses/sizes, robot state, control interface, and available API/residual context.

Output JSON:
- Exactly keys "Reasoning" and "AtomicActions"; no extra text, comments, code fences, or trailing commas.
- "Reasoning": brief interpretation and chosen frames/axes/clearances.
- "AtomicActions": an array with exactly one string. First line starts with the current phase tag; second line is
"Constraints:"; following lines are numbered constraint sentences.

Stage rule:
- The single string is one controller stage. It may list simultaneous constraints, but not sequential motions or sequential
gripper changes.
- If the current subtask is compound, encode only the first controller-feasible component and explain the split issue in
Reasoning.
```

Global rules:
1. Frames/entities
   - Use only observed objects and frames; do not invent entities or surfaces.
   - Every pose, orientation, and translation names its frame: agent/robot base, world, object, or tool frame as appropriate.
   - Do not assert frame/world alignment unless explicitly stated in observations.

2. Axes/orientation
   - Always name signed axes, e.g. tool +z, tool +y, object +x, world +z.
   - In [Approach], approach axis is tool +z. Do not reinterpret [Manipulate] translations as tool +z unless the current subtask says so.
   - Finger axis is tool +y; use it for all finger-axis relations.
   - Axis relation words are only: antiparallel, codirectional, bipolar, orthogonal, or "tool +z points toward the center of <object>".
   - Use axis relation words for axes, not motion direction.

3. Distances/translations
   - Directional translation phrasing, "translate along <signed axis> in <frame> by <positive distance>", is allowed only in [Manipulate] and [Release], not [Approach].
   - Distances are positive; flip the axis sign instead of using negative distances.
   - Non-contact clearance margin must be >= 0.10 m unless tighter is explicit; "small clearance" >= 0.10 m, "large clearance" >= 0.15 m.
   - If size is available, non-contact separation = 0.5*moving clearance-extent + 0.5*target clearance-extent + margin. Clearance-extent is the longest extent unless an axis-specific extent is given.
   - For contact/into-contact, aim for the target frame origin or object center and state contact feedback. Do not define contact using object surfaces/extents.

4. Approach positional goals
   - Non-contact axis approach: reach "<object/frame> origin translated along the chosen/inferred SIGNED object axis by D in object frame"; D >= 0.10 m unless tighter is explicit. Do not express this only as scalar standoff and do not flip the sign.
   - Other non-contact approaches: use a positive standoff from an object reference point.
   - Contact approaches: reach the target frame origin or object center.
   - Never express [Approach] standoff as along a tool axis.

5. Shape/finger rules for grasps
   - Elongated object: tool +y orthogonal to long axis.
   - Box-like object: choose a clamping edge/axis and make tool +y orthogonal to it.
   - Spherical/rotationally symmetric: tool +y is free within symmetry; mention it without over-constraining.
   - Top-down grips: tool +y must not align with tool +z.

Approach label mapping:
- If no approach label appears, assume direct approach: tool +z points toward the object center; add no top-down/leveled/axis orientation constraints.
- If the subtask says keep orientation fixed, do not add approach or shape orientation constraints; state only fixed orientation.
- top-down approach: tool +z antiparallel to world +z.
- leveled approach: tool +z orthogonal to world +z AND points toward the object center.
- direct approach toward the object: tool +z points toward the object center.
- axis approach: tool +z antiparallel to the chosen signed object axis. If the subtask says "appropriate axis approach" and observations give an interaction direction, infer and state that signed axis; if sign is ambiguous, say bipolar.

Phase rules:
[Approach]
- Keep the phase tag.
- Unless orientation is fixed, state tool +z orientation from the approach mapping; if setting up a grasp, also state tool +y shape/finger relation.
- Use the positional-goal rules above.
- State stop conditions using the geometric target and monitored feedback. Use contact/effort feedback only when touching is intended. Do not say hold position/pose unless the subtask explicitly says fixed.

[Manipulate]
- Decide whether the stage changes gripper state with pose fixed, or moves a tool/object while in contact/grasp.
- For gripper-state changes, state which pose is fixed and define firm grasp/release using contact/effort. Later conversion should use object_grasped when success requires a real held object; close_gripper is only mechanical closure.
- For moving while in contact/grasp, use named axes/positive distances or rotations; lift/raise = world +z, lower = world -z, independent of tool orientation. Mention clearances and stop conditions such as geometric completion or effort at a stop.
- For "Align <frame_A> with <frame_B>" without engagement/contact: do not model only center distance; include an axis relation if axes are given, use bipolar if sign is ambiguous, laterally align while maintaining positive axial standoff, and if size is known use the separation formula.
- Never use the same scalar for both "within d" and "minimum standoff >= d" on the same distance; they must be different components or values.
- If reorienting a grasped object, use object-axis relations and preserve grasp/no-slip; do not add absolute tool +z constraints unless explicitly requested.
- Only mention tool +z in [Manipulate] if the subtask says fixed tool orientation or names a tool-axis relation.
- Always state monitored feedback.

[Release]
- For opening/loosening, hold a pose fixed and indicate contact/effort drops toward free-space.
- For retract, use the explicit direction if given; otherwise mirror the most recent approach for the same object/frame: top-down -> world +z; leveled/direct -> tool -z; axis approach -> same signed object axis offset point, not flipped. Use D >= 0.10 m unless tighter is explicit.

```
- State whether tool +z and tool +y preserve previous orientation during retract when relevant.

Self-check before output:
1. Exactly one AtomicActions string and correct phase tag.
2. All poses/orientations/translations name frames and signed axes.
3. [Approach] position goal uses the correct form: axis-offset, positive standoff, or contact target.
4. Size-based standoffs use the extent+margin rule.
5. Output is valid JSON with only "Reasoning" and "AtomicActions".
```

Listing V.6: User template for atomic-stage grounding

```
Top-level task: {task_description}

Current high-level subtask: {current_subtask}
Completed high-level subtasks: {completed}
Pending high-level subtasks: {pending}

Inputs:
* Optional scene image including the robot
* Robot frames / joints: {frames}
* Observed objects: {history}

Technical context:
* robot type: {robot}
* current state: {state}
* control interface: {control}

Ground ONLY the current subtask into exactly ONE detailed natural-language grounded stage requirement.

Rules:
- Preserve the current phase tag and intent.
- Keep the natural-language grounding layer; do not emit TermAPI JSON here.
- The downstream constraint stage treats the entire AtomicActions array as one controller stage, so do not use the array to
describe a sequence.
- The single grounded stage may describe simultaneous constraints, but not multiple sequential motions or sequential gripper
changes.
- If the current subtask is compound, ground only the first controller-feasible component and explain the split issue in
"Reasoning".
- Use the system instructions for frames, tool +z / tool +y, approach labels, translations, clearances, object extents,
finger-vs-shape rules, stop conditions, and effort/contact feedback.
- For [Approach] position goals: non-contact axis approach uses an axis-offset point; other non-contact approaches use
positive standoff; contact approaches reach the target frame origin or object center.
- Specify target geometry and contact/effort conditions. State target distances, standoffs, clearances, and rotation amounts
when needed.

Return exactly one JSON object:
- "Reasoning": how you interpreted the current subtask and applied frame/geometric rules.
- "AtomicActions": an array containing exactly one string formatted as:
  first line = phase + summary;
  second line = "Constraints:";
  following lines = numbered constraint sentences specifying frames, axes, motions, clearances, stop conditions, gripper
state, and feedback signals.
```

4) Typed relation writing: Listing V.7: System context for typed relation writing

```
You translate the CURRENT grounded natural-language action into one flat JSON list of Constraint terms. You do not write code.
The list represents one controller stage. Treat constraints as conjunctive stage requirements: use ‘logic="and"‘ for normal
terms and do not rely on top-level OR unless the runtime/error message explicitly supports it. Encode explicit geometry/effort
stop alternatives within the same relevant term’s ‘condition‘ labels when representable.
```

Listing V.8: User template for typed relation writing

```
### SCOPE & INPUTS

Task
* Current grounded stage requirements: {task_description}
* Completed subtasks, context only: {completed}
* Pending subtasks, context only: {pending}

Observations
* Robot Frames: {frames}
* Observed Objects, visible + history: {history}
* Only refer to names in Robot Frames and Observed Objects; there are no others. "world" is a reserved keyword.

Translate the current grounded natural-language action into a flat JSON list of Constraint terms. Do not write code.
Return only the fields in the schema below. Preserve target distances, offsets,
rotation angles, frames, and contact conditions.
```

STAGE SEMANTICS

The Constraints list represents ONE controller stage.
* Cover all required components of the stage; do not drop a component because another is easier to encode.
* Stage requirements are conjunctive unless a runtime/error message explicitly says top-level OR is supported.
* For normal terms, set `logic` to `"and"`.
* Do not rely on top-level `logic="or"` for alternatives. If the text gives a supported stop alternative such as geometric progress OR manipulator_high_effort, encode it in the same relevant term by including both condition labels, e.g. `["geometry", "manipulator_high_effort"]`. If the alternative cannot be represented, keep the required representable term and note the omitted alternative in Reasoning.

Condition labels describe satisfaction/feedback categories:
* `geometry`: pose, motion, orientation, or freeze constraints.
* `gripper_opened` / `gripper_closed`: intended gripper state metadata; not arm contact effort. `gripper_closed` alone does not prove a grasp.
* `manipulator_high_effort` / `manipulator_low_effort`: arm/TCP interaction effort for pushing, pressing, pulling, or sliding; not finger squeeze.

TERM API

Use ONLY these ops and EXACTLY these args keys:

```json
{
  "reach_point": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "offset_from_origin_xyz": [float, float, float]
    }
  },
  "directional_move": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "axis": "+x | -x | +y | -y | +z | -z",
      "travel_distance_m": float
    }
  },
  "axis_align": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "controlled_axis": "+x | -x | +y | -y | +z | -z",
      "reference_axis": "+x | -x | +y | -y | +z | -z",
      "alignment": "codirectional | antiparallel | bipolar | orthogonal"
    }
  },
  "point_toward": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "controlled_axis": "+x | -x | +y | -y | +z | -z",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "offset_from_origin_xyz": [float, float, float]
    }
  },
  "directional_rotate": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "rotation_axis": "+x | -x | +y | -y | +z | -z",
      "rotation_angle_rad": float
    }
  },
  "freeze_position": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME"
```

```
    }
  },
  "freeze_orientation": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME"
    }
  },
  "open_gripper": {"args": {}},
  "close_gripper": {"args": {}},
  "object_grasped": {"args": {}}
}
```

NAME must be a valid robot frame or object/object-attached frame from Observations. Do not add or omit args keys.

FIELD MEANINGS

* `controlled_entity="robot_frame"` requires `controlled_target` to be a robot frame.
* `controlled_entity="gripped_object"` requires `controlled_target` to be the object/object-attached frame currently in the gripper.
* `reference_basis` is the coordinate system used for vectors/axes; `reference_origin` is the pose defining that basis origin/axes.
* `offset_from_origin_xyz` is applied at `reference_origin` in `reference_basis`.
* `_m` values are meters; `_rad` values are radians.

OPERATOR RULES

* `reach_point`: reach the specified target point; path is unconstrained. Use for concrete targets, pre-grasps, standoffs, and approach-side/axis offsets.
* `directional_move`: move strictly along one signed axis by positive `travel_distance_m`. Use only when the text explicitly specifies axis/direction plus distance; otherwise use `reach_point`.
* `axis_align`: align axes with codirectional, antiparallel, bipolar, or orthogonal relation. Use `bipolar` when either direction is acceptable.
* `point_toward`: make a controlled signed axis aim at a target point.
* `directional_rotate`: rotate about a signed axis by an explicit angle.
* Do not combine `directional_rotate` with `axis_align` in the same stage when the `axis_align` constrains or references the frame being rotated. Put approach/pre-grasp axis alignment in an earlier stage; the rotation stage should keep the finite `directional_rotate` goal.
* `freeze_position`: keep current position fixed; do not use as a target-reaching goal.
* `freeze_orientation`: keep current orientation fixed; do not use as a goal-alignment operator. If the stage establishes a new axis relation, emit `axis_align` or `directional_rotate`; `freeze_orientation` alone is invalid for that.
* `open_gripper`: gripper open/released at success.
* `close_gripper`: mechanical finger closure only when a real held object is not required.
* `object_grasped`: object must be physically held; use for grasp, hold, keep holding, and any later moving/pulling/carrying/aligning stage that assumes the object remains in the gripper. Do not emit both `close_gripper` and `object_grasped` in one list.

General constraints:
1. At most one position op in the whole list: `reach_point` OR `directional_move` OR `freeze_position`.
2. Every constraint must be justified by the current stage requirements.
3. Describe the intended contact/fit precisely through geometry and conditions.
4. If a required clause cannot be represented, omit that clause from Constraints and state it in Reasoning.

REQUIRED OUTPUT

Return exactly one JSON object:

```json
{
  "Reasoning": "string",
  "Constraints": [
    {
      "logic": "and | or",
      "condition": ["geometry", "gripper_opened", "gripper_closed", "manipulator_high_effort", "manipulator_low_effort"],
      "op": "reach_point | directional_move | axis_align | point_toward | directional_rotate | freeze_position | freeze_orientation | open_gripper | close_gripper | object_grasped",
      "args": {}
    }
  ]
}
```

Reasoning briefly explains success, each constraint, any condition-label alternative, and any unrepresented clause.

5) Typed-relation correction: Listing V.9: System context for typed-relation correction

```
You correct the PREVIOUS grounded output for the CURRENT stage after a failure. You do not write code.
Use the error message as the primary diagnostic signal. Make the smallest change needed, keeping correct constraints verbatim.
The corrected Constraints list is one controller stage: prefer `logic="and"`; do not rely on top-level OR unless the runtime/
```

```
error message explicitly supports it. Encode representable stop alternatives inside the same term's ‘condition‘ labels.
```

Listing V.10: User template for typed-relation correction

```
### SCOPE & INPUTS

Task
* Current grounded stage requirements: {task_description}
* Completed subtasks, context only: {completed}
* Pending subtasks, context only: {pending}

Observations
* Robot Frames: {frames}
* Observed Objects, visible + history: {history}
* Only refer to names in Robot Frames and Observed Objects; there are no others. "world" is a reserved keyword.

Previous grounded output to correct:
* {previous_grounding}

Current error message, primary signal:
* {error_message}

### YOUR JOB

Return only the fields in the schema below. Correct geometry/contact conditions
while preserving the intended target and required physical result.

Diagnose the failure, then return corrected Constraint JSON that:
1. Uses only TermAPI ops and exactly their allowed args keys.
2. References only valid observed names.
3. Preserves all intended success criteria for the current stage.
4. Changes as little as possible: edit only fields implicated by the error, schema/API violations, missing coverage,
unsupported top-level OR, or the one-position-op rule. Keep correct fields verbatim.

### STAGE SEMANTICS AND REPAIR RULES

The Constraints list represents ONE controller stage.
* Cover all required components; do not drop a component because another is easier to encode.
* Stage requirements are conjunctive unless a runtime/error message explicitly says top-level OR is supported.
* For normal terms, set ‘logic‘ to ‘"and"‘.
* Do not rely on top-level ‘logic="or"‘ for alternatives. If the text gives a supported stop alternative such as geometric
progress OR manipulator_high_effort, encode it in the same relevant term by including both condition labels, e.g.
‘["geometry", "manipulator_high_effort"]‘. If the alternative cannot be represented, keep the required representable term and
note the omitted alternative in Diagnosis or Reasoning.
* Across the whole list, keep at most one position op: ‘reach_point‘ OR ‘directional_move‘ OR ‘freeze_position‘. If violated,
keep the one that best matches the explicit positional requirement and mention the dropped op in Diagnosis.
* Remove only exact duplicates: same ‘op‘ and identical ‘args‘; merge ‘condition‘ labels by unique union. Do not collapse
merely similar constraints.
* ‘gripper_opened‘ / ‘gripper_closed‘ are gripper-state metadata, not arm contact effort. ‘manipulator_high_effort‘ /
‘manipulator_low_effort‘ are arm/TCP contact effort, not finger squeeze. Use ‘object_grasped‘ when a real held object is
required; do not represent contact/effort with gripper ops.

### TERM API

Use ONLY these ops and EXACTLY these args keys:

‘‘‘json
{
  "reach_point": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "offset_from_origin_xyz": [float, float, float]
    }
  },
  "directional_move": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "axis": "+x | -x | +y | -y | +z | -z",
      "travel_distance_m": float
    }
  },
  "axis_align": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
```

```
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "controlled_axis": "+x | -x | +y | -y | +z | -z",
      "reference_axis": "+x | -x | +y | -y | +z | -z",
      "alignment": "codirectional | antiparallel | bipolar | orthogonal"
    }
  },
  "point_toward": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "controlled_axis": "+x | -x | +y | -y | +z | -z",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "offset_from_origin_xyz": [float, float, float]
    }
  },
  "directional_rotate": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME",
      "reference_basis": "world | object | robot_frame",
      "reference_origin": "NAME | world",
      "rotation_axis": "+x | -x | +y | -y | +z | -z",
      "rotation_angle_rad": float
    }
  },
  "freeze_position": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME"
    }
  },
  "freeze_orientation": {
    "args": {
      "controlled_entity": "robot_frame | gripped_object",
      "controlled_target": "NAME"
    }
  },
  "open_gripper": {"args": {}},
  "close_gripper": {"args": {}},
  "object_grasped": {"args": {}}
}
```

NAME must be a valid robot frame or object/object-attached frame from Observations. Do not add or omit args keys.

FIELD AND OPERATOR RULES

* `controlled_entity="robot_frame"` requires a robot-frame target; `controlled_entity="gripped_object"` requires an object/object-attached frame currently in the gripper.
* `reference_basis` is the coordinate system for vectors/axes; `reference_origin` defines the pose. `_m` = meters; `_rad` = radians.
* `reach_point`: concrete target point, pre-grasp, standoff, or approach offset; path unconstrained.
* `directional_move`: strictly along one signed axis by positive distance; only when axis/direction plus distance is explicit. Otherwise use `reach_point`.
* `axis_align`: axis relation; use `bipolar` when either direction is acceptable.
* `point_toward`: make a controlled axis aim at a target point.
* `directional_rotate`: explicit rotation about an axis by an angle.
* Do not combine `directional_rotate` with `axis_align` in the same stage when the `axis_align` constrains or references the frame being rotated. Put approach/pre-grasp axis alignment in an earlier stage; the rotation stage should keep the finite `directional_rotate` goal.
* `freeze_position`: keep current position fixed; not a target-reaching goal.
* `freeze_orientation`: keep current orientation fixed; not goal alignment. If the stage establishes a new axis relation, corrected output must contain `axis_align` or `directional_rotate`.
* `open_gripper`: open/released. `close_gripper`: mechanical closure only. `object_grasped`: real held object for grasp/hold/keep holding and later carrying/pulling/manipulating. Do not emit both `close_gripper` and `object_grasped`.

REQUIRED OUTPUT

Return exactly one JSON object:

```json
{
  "Diagnosis": "string",
  "Reasoning": "string",
  "Constraints": [
    {
```

```
      "logic": "and | or",
      "condition": ["geometry", "gripper_opened", "gripper_closed", "manipulator_high_effort", "manipulator_low_effort"],
      "op": "reach_point | directional_move | axis_align | point_toward | directional_rotate | freeze_position | freeze_
orientation | open_gripper | close_gripper | object_grasped",
      "args": {}
    }
  ]
}
‘‘‘

Diagnosis ties the failure to the error message. Reasoning explains success, each constraint, any condition-label alternative,
and any unrepresented clause. Constraints are the corrected ordered list.
```

B. Learning and calibration prompts

These templates expose scalar choices and measured evidence without permitting the model to invent a new operator, stage, reward implementation, or task evaluator. Initial selection adjusts weights without rollout evidence. The BC-prior diagnostic variant then uses a deterministic prior rollout, not residual-RL training statistics, to identify components blocking progress. Its system prompt concatenates the shared supervision contract, the source's BC-specific note, and the stagewise weight-adjustment instructions. The separate residual-authority pair receives learning traces. None of these templates receives or edits the code-owned tolerance fields described in Appendix II-B.

An optional video summarizer can produce evidence, but it has no write authority over the program. Its prose suggestions are constrained by the system prompts below, which define the editable fields, numeric ranges, and output schemas. Accordingly, this appendix includes the formation and active initialization/calibration templates, while excluding the deprecated stage/global-runner variants, video-captioning instructions, and run-specific transcripts.

1) Initial reward-weight selection: Listing V.11: System prompt for initial reward-weight selection

```
You are a supervisor for a reinforcement learning agent. The agent is training on a multi-stage task where each stage has its
own set of reward components.

These reward components are based on cost terms that have been effective in prior optimal control settings. Do not add,
remove, or redefine reward components.

Each reward component *i* defines:

1. a goal requirement,
2. a shaping reward function that increases as the agent gets closer to meeting that requirement, and
3. a constraint check that measures whether the goal requirement has been met.

A stage is considered complete when all component constraints are satisfied.

Task names, object labels, stage numbers, and component indices identify the
provided data; they do not determine which component deserves more weight.
Base each decision on the current physical requirement and, when provided,
measured evidence. Do not infer a stored solution from a familiar name.

Short rule examples, not task solutions:
- Weights [1, 2] and [2, 4] express the same relative preference after normalization.
- A pose hold during release and a pose requirement that validates the final
  object state can have different roles even when both use the same operator.

Physical roles used by the weight rules:
- Broad body: roughly solid/boxlike/round grasp cross-section >= 0.05 m,
  without a narrow axis or receiver-fit requirement.
- Small solid body: cross-section above 0.03 m and below 0.05 m; size alone
  does not make orientation mission-critical.
- Small/slender: cross-section <= 0.03 m, or a long/thin/axis-critical shape.
- Release guard: robot/tool pose held while opening after a supported result;
  distinguish this from a component checking the object's final validity.
- Precision progress: the traveled amount itself validates current contact,
  fit, or actuation; free-space waypoints are not precision progress.

You initialize reward-component Weights for ONE stage before RL.
Adjust ONLY reward-component Weights.

Do NOT change reward/constraint definitions, indices, ordering, logic
structure, targets, distances, or any other configuration.

Use only the provided task/stage semantics, observed objects, adjacent-stage
context, component explanations, and current weights.
Do NOT assume rollout statistics, success rates, reward traces, or hidden task
facts.

Weights:
- Set weights by semantic role, not performance. Reward is normalized by sum(weights), so tune relative ratios.
- Default init-sweep weight is 1.0. Use simple ratios.
- Broad free-space progress, broad-body ordinary pose holds, broad-body top-down/side orientation maintenance, and
non-critical maintenance terms usually stay in [1.0, 1.5].
```

```
- For broad_body tasks with no small/slender, receiver/opening, constrained-feature, or final-object-validity requirement,
avoid weights above 2.0. Grasp/object-state may be 2.0 if losing the state invalidates all progress, but do not inflate it to
2.5-3.0 on init sweep.
- Direct fit/contact, small-cross-section grasp-lock, constrained-feature manipulation, precision directional progress,
mission-critical angular constraints, and final object validity may use [1.5, 3.0] depending on whether losing that component
invalidates the stage.
- Open-gripper setup/contact orientation terms and broad sliding-pull orientation terms are usually [1.0, 1.5] and should not
be weighted as mission-critical angular constraints unless the current stage truly requires angular fit, insertion, final
object validity, or a force/torque axis.
- Gripper/object state components are important when they define contact, release, or grasp validity; usually use 1.5, use 2.0
for broad-body state validity when losing state invalidates progress, and reserve 2.5-3.0 for small/slender, receiver/opening,
constrained-feature, or final-object-validity cases.
- Release_guard robot/tool pose holds usually stay in [1.0, 1.5]; do not give them narrow-feature/manipulation-level weights
unless the stage explicitly says the held tool pose itself is the contact-validity condition.
- Avoid weight inflation: do not set every core component above 2.0 on the init sweep, and do not make broad-body orientation/
maintenance terms higher than the primary progress/contact term.
- weight must be finite and in [0.1, 10.0]. Correct any out-of-bound value.

Return ONLY one valid JSON object with exactly:
"Diagnosis", "Reasoning", "Weights"

"Diagnosis": briefly list the semantic role of each tunable component.
"Reasoning": briefly justify the labels and relative weight ratios.
"Weights" MUST contain EXACTLY the same indices as current weights.
```

Listing V.12: User template for initial reward-weight selection

```
Overall task description: {task_description}

Detectable objects: {detectable_objects}

Stage index/total number of stages: {stage_id}/{stage_count}

Stage description: {atomic_action}

Current subtask constraint ops/args:
{current_constraint_ops}

Adjacent subtask context (for transition semantics only; tune ONLY the current stage):
Previous subtask:
{previous_stage_context}

Next subtask:
{next_stage_context}

Index-ordered reward component explanations:
{constraints_explanations}

Current weights:
{current_weights}
```

2) BC-prior-informed reward-weight adjustment: Listing V.13: System prompt for BC-prior-informed weight adjustment

```
You are a supervisor for a reinforcement learning agent. The agent is training on a multi-stage task where each stage has its
own set of reward components.

These reward components are based on cost terms that have been effective in prior optimal control settings. Do not add,
remove, or redefine reward components.

Each reward component *i* defines:

1. a goal requirement,
2. a shaping reward function that increases as the agent gets closer to meeting that requirement, and
3. a constraint check that measures whether the goal requirement has been met.

A stage is considered complete when all component constraints are satisfied.

Task names, object labels, stage numbers, and component indices identify the
provided data; they do not determine which component deserves more weight.
Base each decision on the current physical requirement and, when provided,
measured evidence. Do not infer a stored solution from a familiar name.

Short rule examples, not task solutions:
- Weights [1, 2] and [2, 4] express the same relative preference after normalization.
- A pose hold during release and a pose requirement that validates the final
  object state can have different roles even when both use the same operator.

Physical roles used by the weight rules:
- Broad body: roughly solid/boxlike/round grasp cross-section >= 0.05 m,
  without a narrow axis or receiver-fit requirement.
- Small solid body: cross-section above 0.03 m and below 0.05 m; size alone
  does not make orientation mission-critical.
```

```
- Small/slender: cross-section <= 0.03 m, or a long/thin/axis-critical shape.
- Release guard: robot/tool pose held while opening after a supported result;
  distinguish this from a component checking the object's final validity.
- Precision progress: the traveled amount itself validates current contact,
  fit, or actuation; free-space waypoints are not precision progress.

You are doing an initial weight acquisition pass from a deterministic BC-prior rollout.
The rollout statistics are not RL-training results; they show where the BC prior times out.
The timeout/stuck component rates are the key evidence: high not_satisfied means that
component blocked the BC prior for envs that reached and timed out in that stage.

Adjust ONLY reward-component Weights for the current stage.

You tune ONE stage by adjusting ONLY reward-component Weights.

Do NOT change reward/constraint definitions, indices, ordering, logic structure, or any other configuration.

Use stage semantics and population stats.
Reward is normalized by sum(weights), so tune relative ratios.

For each component, identify its semantic role and main blocker:
- poor progress / control
- weight tradeoff
- required state not maintained

Judge meaningful completion by whether the resulting state is actually suitable for the intended interaction, not merely
whether predicates became true.
Do NOT use weighted reward magnitude alone as evidence of geometric closeness.

Return ONLY one valid JSON object (no markdown, no code fences, no extra text).
Top-level keys MUST be EXACTLY:
  "Diagnosis", "Reasoning", "Weights"

- "Weights" MUST contain EXACTLY the same indices as the provided current weights.
- Do NOT add/remove/rename indices.

Weights:
- finite, final value in [0.1, 10.0]
- per-index step limit: new_w[i] in [0.5*old_w[i], 2.0*old_w[i]]
- if a current weight lies outside the hard bounds, you MUST correct it into the hard-bound interval on this pass

Optimization priority:
1) Increase meaningful stage completion.
2) Preserve semantic validity of the interaction.
3) Improve precision only if (1) and (2) do not worsen.
4) Improve efficiency only if (1), (2), and (3) do not worsen.
```

Listing V.14: User template for BC-prior-informed weight adjustment

```
Overall task description: {task_description}

Detectable objects: {detectable_objects}

Stage index/total number of stages: {stage_id}/{stage_count}

Stage description: {atomic_action}

Index-ordered reward component explanations:
{constraints_explanations}

Current weights:
{current_weights}

========================
END-TO-END PERFORMANCE (ALL ENVS)
========================
{end_to_end_feedback}

Stage satisfaction times:
{stage_satisfaction_times}

Per-component constraint satisfaction rates:
{constraint_satisfaction_rates}

Per-component reward values (multiplied by weights):
{reward_values}
```

3) Residual-authority calibration: Listing V.15: System prompt for residual-authority calibration

```
You are a supervisor for a reinforcement learning agent. The agent is training on a multi-stage task where each stage has its
own set of reward components.

These reward components are based on cost terms that have been effective in prior optimal control settings. Do not add,
remove, or redefine reward components.
```

Each reward component *i* defines:

1. a goal requirement,
2. a shaping reward function that increases as the agent gets closer to meeting that requirement, and
3. a constraint check that measures whether the goal requirement has been met.

A stage is considered complete when all component constraints are satisfied.

Task names, object labels, stage numbers, and component indices identify the
provided data; they do not determine which component deserves more weight.
Base each decision on the current physical requirement and, when provided,
measured evidence. Do not infer a stored solution from a familiar name.

Short rule examples, not task solutions:
- Weights [1, 2] and [2, 4] express the same relative preference after normalization.
- A pose hold during release and a pose requirement that validates the final
  object state can have different roles even when both use the same operator.

Physical roles used by the weight rules:
- Broad body: roughly solid/boxlike/round grasp cross-section >= 0.05 m,
  without a narrow axis or receiver-fit requirement.
- Small solid body: cross-section above 0.03 m and below 0.05 m; size alone
  does not make orientation mission-critical.
- Small/slender: cross-section <= 0.03 m, or a long/thin/axis-critical shape.
- Release guard: robot/tool pose held while opening after a supported result;
  distinguish this from a component checking the object's final validity.
- Precision progress: the traveled amount itself validates current contact,
  fit, or actuation; free-space waypoints are not precision progress.

You tune residual-RL agent hyperparameters for a manipulation task.

Allowed edits only: params.env.clip_actions, params.config.residual_rl.residual_bounds,
params.config.residual_rl.clip_composed_actions, params.config.residual_rl.alpha_final,
and params.config.residual_rl.alpha_warmup_epochs. All reward, constraint, task, stage,
robot, and BC-prior settings are fixed.

The prompt gives full detectable_objects JSON, short stage labels, action-dimension labels,
outcome/stage-satisfaction stats, compact TensorBoard scalar summaries, and when available
a 240x240 image of Episode/Hariti/success_rate_window.

Collapse definition:
- A collapse requires the latest post-warmup success_rate_window to be at least 50% lower
  than the previous post-warmup peak: peak_to_latest_drop_fraction >= 0.5.
- Do not classify smaller late dips as collapse. If collapse_after_peak is false, explicitly
  mention that the curve may be a normal run unless other evidence strongly indicates instability.
- Inspect the attached success_rate_window plot, when present, and use the visual trend to
  sanity-check the numeric summary before deciding.

The scalar summary includes:
- Episode/Hariti/success_rate_window after critic_warmup_epochs only, including first valid value,
  stable peak, collapse onset, latest value, peak-to-latest drop, and 9 or fewer trend samples
- Episode/Hariti/false_complete_rate
- Episode/Hariti/timeout_rate
- Hariti/rollout_step_reward_mean_window/iter
- Hariti/rollout_epoch_step_reward_mean/iter

Use the action-dimension labels when editing residual_bounds. residual_bounds[i] controls the
residual range for action dimension i.

Decision policy:
- First read the success_rate_window trend samples and inspect the attached plot, if present.
- Use collapse_onset_step, stable_peak_step, latest_post_warmup, and peak_to_latest_drop_fraction
  to decide whether the 50% collapse rule is actually met.
- If collapse_after_peak is true under the 50% rule, slow down residual authority by increasing
  alpha_warmup_epochs, up to 1000.
- If collapse happens near or after the current warmup limit, also reduce alpha_final or tighten
  the action dimensions most likely causing drift.
- If collapse_after_peak is false, do not reduce aggression solely because the latest value is
  slightly below the peak; say it may be a normal run and use outcome stats, plot shape, false
  completions, timeouts, and rewards to choose minimal changes.
- Treat enabling clip_composed_actions, reducing clip_actions, reducing alpha_final, and tightening
  residual_bounds as restrictive safety moves. Use them only when there is clear collapse,
  instability, reward-proxy drift, high false-complete rate, or direct action-clipping evidence.
- If there is no clear collapse/instability, do not make action clipping more restrictive. Preserve
  the current clip_composed_actions value, or set it false/unlock clipping if stable success appears
  limited by composed-action clipping.
- If success is low without a peak/drop pattern and rewards are flat, allow a small amount more
  correction: consider increasing clip_actions up to its hard limit, loosening the relevant
  residual_bounds within bounds, or keeping alpha_final high rather than reducing it.
- If reward improves but success_rate_window drops sharply, treat that as reward-proxy drift and make the residual safer.
- Change the fewest hyperparameters needed; prefer alpha_warmup_epochs/alpha_final changes before broad residual_bounds
changes when the signature is alpha-collapse.

```
Hard numeric bounds:
- clip_actions must be finite and in [0.5, 2.0].
- alpha_final must be finite and in [0.05, 1.5].
- alpha_warmup_epochs must be an integer in [0, 1000].
- residual_bounds must be the same length as the current list.
- each residual_bounds value must be finite and in [0.02, 1.0].
- Per-iteration step limit: each numeric value should stay within [0.5 * current, 1.5 * current],
  unless the current value is outside the hard numeric bounds and must be corrected.

Return ONLY one valid JSON object with exactly:
"Diagnosis", "Reasoning", "Hyperparameters"

"Diagnosis": summarize whether the run appears under-corrected, over-corrected, unstable, or stable.
"Reasoning": justify each changed hyperparameter and why unchanged values should remain unchanged.
"Hyperparameters" MUST contain exactly these keys:
  "clip_actions", "residual_bounds", "clip_composed_actions", "alpha_final", "alpha_warmup_epochs"
Do not add, remove, or rename keys.
```

Listing V.16: User template for residual-authority calibration

```
Task:
{task_description}

Detectable objects:
{detectable_objects}

Stages:
{stage_labels}

Action mode:
{action_mode}

Action dimension labels:
{action_dimension_labels}

Current residual-RL hyperparameters:
{current_hyperparameters}

Outcome stats:
{outcome_stats}

Stage satisfaction summary:
{stage_satisfaction_summary}

Exact TensorBoard scalar summaries:
{reward_success_event_summary}

Success-rate plot:
{success_rate_plot_summary}
```